

TCC · 56 PÁGINAS · DOCUMENTO UNIFICADO

Controla.AI, Organização do Sistema e Arquitetura do Banco de Dados

Explicação completa e didática de **como o software é organizado** (pastas, servidor, agente de inteligência artificial, WhatsApp, painel web e publicação em nuvem) e de **como o PostgreSQL guarda e conecta os dados**, com uma página inteira dedicada a cada uma das 16 tabelas.

Fontes oficiais incorporadas integralmente neste documento

- 1) **TCC_DOCUMENTACAO.md**, visão geral, camadas, estrutura de pastas, rotas, fluxos, agente de IA, deploy (49.924 caracteres)
- 2) **CONEXOES_BANCO_DADOS.md**, as 18 ligações, cardinalidade, CASCADE/SET NULL, entradas e saídas por tabela (26.223 caracteres)
- 3) **ARQUITETURA_BANCO_COMPLETA.md**, diagrama ER, todas as colunas, tipos e chaves das 16 tabelas (14.380 caracteres)
- 4) **Diagramas PNG**, arquitetura-banco-diagrama.png e arquitetura-banco-detallhes.png, reproduzidos em tamanho de página

Parte A · O sistema (pág. 3 a 24)

O que o produto faz, as seis camadas da arquitetura, a estrutura de pastas completa, o servidor Fastify arquivo a arquivo, o agente de inteligência artificial, a integração com o WhatsApp, o painel web e a publicação em nuvem.

Parte B · O banco (pág. 25 a 48)

Como o código se transforma em tabelas, como ler chaves e regras de exclusão, o mapa dos seis grupos, as 18 ligações uma a uma, os dois diagramas oficiais e **uma página inteira para cada uma das 16 tabelas**.

Parte C · Os fluxos (pág. 49 a 54)

Quatro caminhos ponta a ponta com as tabelas tocadas em cada etapa: registrar um gasto pelo WhatsApp, entrar no painel, criar uma meta e responder “quanto gastei?”. Mais a origem de cada indicador da tela.

Fechamento (pág. 55 e 56)

Checklist com as perguntas mais prováveis da banca e a resposta curta de cada uma, roteiro sugerido de demonstração, lista das fontes utilizadas e conclusão do trabalho.

16

tabelas detalhadas

18

chaves estrangeiras

147

colunas explicadas

6

grupos de domínio

\$

Sumário, o que há em cada página

Quatro partes: o sistema, a ponte, o banco e os fluxos

Este documento foi escrito para ser lido por qualquer pessoa, inclusive quem nunca programou. A **Parte A** explica o software: o que ele faz, como os arquivos estão organizados e como cada pedaço conversa com os outros. A **Parte B** explica o banco de dados: o que é uma chave, como as tabelas se ligam e o que cada coluna guarda. A **Parte C** costura as duas: mostra o caminho completo de uma frase no WhatsApp até o gráfico do painel.

Páginas	Parte	Conteúdo detalhado	Fonte principal
1 a 2	Abertura	Capa e este sumário	-
3 a 4	A · Produto	O que é o Controla.AI, para quem serve, o que resolve, glossário de tecnologias em linguagem simples	TCC_DOCUMENTACAO §1
5 a 6	A · Arquitetura	As seis camadas do sistema e o diagrama do caminho que um dado percorre	TCC_DOCUMENTACAO §2
7 a 9	A · Pastas	Estrutura de pastas: raiz, backend/src, backend/api, backend/whatsapp, frontend, drizzle, scripts	TCC_DOCUMENTACAO §3 e §3.1
10 a 12	A · Backend	Servidor Fastify arquivo a arquivo: boot, autenticação, rotas de CRUD e rotas estendidas	TCC_DOCUMENTACAO §5
13 a 15	A · Inteligência	Agente unificado, parser da OpenAI, processamento de mídia e cada tipo de consulta (queryType)	TCC_DOCUMENTACAO §6 e §15
16 a 18	A · WhatsApp	Baileys: conexão por QR, pipeline completo da mensagem, proteções contra duplicidade e keep-alive	TCC_DOCUMENTACAO §7
19 a 21	A · Frontend	Páginas do painel React, cliente HTTP, middleware e funções serverless de autenticação na Vercel	TCC_DOCUMENTACAO §8
22 a 24	A · Publicação	Railway, Vercel, variáveis de ambiente, health check e comandos de publicação	TCC_DOCUMENTACAO §11 e §12
25	B · Ponte	Como o código TypeScript se transforma em tabelas reais no PostgreSQL	Drizzle ORM + schema.ts
26	B · Alfabetização	Como ler PK, FK, 1:1, 1:N, 0:1, CASCADE e SET NULL sem ser da área	CONEXOES, Como ler
27	B · Mapa	As 16 tabelas organizadas em 6 grupos, com users no centro	CONEXOES, Mapa rápido
28 a 29	B · Ligações	As 18 chaves estrangeiras, uma a uma, explicadas em parágrafo de leigo	CONEXOES, Lista das 18
30 a 31	B · Diagramas	Os dois PNGs oficiais da modelagem, em tamanho de página inteira	ARQUITETURA §4
32	B · Modelagem	Decisões de projeto: UUID, JSONB, numeric, enums e integridade referencial	ARQUITETURA §1 a §3
33 a 48	B · Tabelas	Uma página inteira por tabela: analogia, função no app, todas as colunas, entradas/saídas de FK, consulta SQL e exemplo real	ARQUITETURA §3 + CONEXOES
49 a 52	C · Fluxos	Quatro caminhos ponta a ponta: gasto pelo WhatsApp, login no painel, criação de meta e a pergunta “quanto gastei?”	TCC_DOCUMENTACAO §4
53 a 54	C · Indicadores	De qual tabela sai cada número do dashboard e de cada resposta do assistente	insights.ts + schema
55 a 56	Fechamento	Checklist de perguntas da banca com resposta pronta, lista de fontes e conclusão	-

Como este documento foi gerado
Ele não foi digitado à mão. Um script em TypeScript (`backend/scripts/generate-ORGANIZACAO-SISTEMA-htm1.ts`) lê os três documentos oficiais, embute os diagramas PNG e monta este HTML; um segundo script usa o navegador Chrome em modo automático para imprimir tudo em PDF A4. Na prática, atualizar a documentação é rodar dois comandos, e não refazer a diagramação.

A

O que é o Controla.AI

Explicado como para quem nunca usou o sistema

Imagine que você quer saber para onde vai o seu dinheiro, mas detesta planilhas. Você não quer abrir aplicativo, escolher categoria em lista suspensa, digitar valor em campo formatado e clicar em salvar, porque, sendo honesto, ninguém faz isso por muito tempo. O **Controla.AI** nasceu dessa constatação simples: o controle financeiro falha não por falta de ferramenta, mas porque registrar gasto dá trabalho.

A solução do projeto é tirar o formulário do caminho. O usuário manda uma mensagem no **WhatsApp**, o aplicativo que ele já usa dezenas de vezes por dia, escrevendo do jeito que falaria com um amigo: “gastei 45 no almoço”. Do outro lado, uma inteligência artificial lê a frase, entende que houve uma despesa de R\$ 45 relacionada a alimentação, grava no banco de dados e responde confirmando. O esforço do usuário foi escrever cinco palavras.

Na prática, existem **dois jeitos de usar** o mesmo sistema. No WhatsApp, a conversa é rápida e informal, serve para registrar na hora em que o gasto acontece e para perguntar coisas soltas (“posso gastar 300 este mês?”). No **painel web**, a experiência é visual, gráficos, listas, barras de progresso de metas e importação de extrato em PDF. Os dois caminhos escrevem e leem **o mesmo banco de dados**, e é essa decisão de arquitetura que evita o pior defeito possível num app financeiro: dois lugares mostrando números diferentes.

Quem usa
Pessoa física que quer controlar o próprio dinheiro sem virar especialista em planilhas. Não exige conhecimento contábil.

O que ele resolve
O atrito do registro. Se registrar é fácil, o histórico fica completo; com histórico completo, todo indicador passa a ser confiável.

Como se diferencia
A entrada é conversa em linguagem natural, não formulário. E aceita texto, áudio, foto de comprovante e extrato em PDF.

AS QUATRO PORTAS DE ENTRADA DE INFORMAÇÃO

Porta de entrada	Como o usuário usa	O que o sistema faz por trás	Onde termina
Texto no WhatsApp	Escreve “gastei 45 no almoço” na conversa com o número oficial	O Baileys recebe a mensagem, o sistema identifica o telefone e a IA interpreta a frase em JSON estruturado	Uma linha em <code>transactions</code>
Áudio no WhatsApp	Grava um áudio dizendo o gasto, sem digitar nada	O modelo Whisper da OpenAI transcreve o áudio para texto e o texto segue o mesmo caminho do item acima	Uma linha em <code>transactions</code>
Foto de comprovante	Fotografa a nota fiscal ou o comprovante do Pix	O modelo de visão lê a imagem, extrai valor e estabelecimento e devolve texto para o interpretador	Uma linha em <code>transactions</code>
Extrato em PDF	Envia o extrato do banco pelo painel web	A biblioteca pdf-parse extrai o texto, a IA identifica os lançamentos e cria vários de uma vez	Várias linhas + registro em <code>document_imports</code>
Painel web	Cadastra manualmente, edita, cria metas e vê gráficos	As rotas REST validam os dados com Zod e gravam pelo Drizzle ORM	<code>transactions</code> , <code>goals</code> , <code>budgets</code>
Contas fixas	Cadastra o aluguel uma vez e esquece	O sistema guarda a recorrência e lança automaticamente na data de vencimento	<code>recurring_transactions</code> → <code>transactions</code>

A regra que sustenta tudo

Toda informação com valor financeiro passa pelo servidor e termina no PostgreSQL. O WhatsApp é apenas um carteiro que entrega a mensagem; o painel web é apenas uma vitrine que mostra o resultado. Nenhum dos dois guarda dado por conta própria. Na prática, existe **uma única fonte de verdade**: se o número está no banco, ele é o número, em qualquer tela, em qualquer canal.



O que o sistema faz e com que tecnologias

Glossário sem jargão + exemplo de diálogo real

Antes de entrar na organização dos arquivos, vale traduzir os nomes técnicos que vão aparecer em todas as próximas páginas. A tabela abaixo faz isso com uma analogia para cada tecnologia, a ideia é que, ao encontrar “Fastify” na página 10, você já saiba do que se trata sem voltar.

GLOSSÁRIO, O QUE É CADA TECNOLOGIA, EM LINGUAGEM DE LEIGO

Tecnologia	Camada	O que é, sem jargão
React + Vite + Tailwind	Frontend	É a vitrine: as telas que o usuário vê e clica no navegador. React monta a tela, Vite acelera o desenvolvimento e Tailwind cuida da aparência.
Node.js + Fastify	Backend	É o balcão de atendimento. Node roda o código no servidor e Fastify é quem recebe os pedidos (“me dê as transações de setembro”) e responde rápido.
TypeScript	Todo o projeto	É o JavaScript com conferência antes de rodar. Se um campo deveria ser número e recebe texto, o erro aparece na escrita do código, não para o usuário.
PostgreSQL	Banco de dados	É o arquivo-morto organizado: guarda tudo em tabelas com regras rígidas que impedem dado inconsistente de entrar.
Drizzle ORM	Ponte	É o tradutor entre o código e o banco. Escreve-se TypeScript e ele gera o SQL correto, sem risco de erro de digitação em consulta.
OpenAI (GPT, Whisper, visão)	Inteligência	É o intérprete. GPT entende frases soltas, Whisper transforma áudio em texto e o modelo de visão lê imagens de comprovantes.
Baileys	WhatsApp	É a biblioteca que conecta um número de WhatsApp ao sistema pelo mesmo QR Code do WhatsApp Web.
Railway	Nuvem	É o computador sempre ligado que hospeda o servidor, o robô do WhatsApp e o banco de dados.
Vercel	Nuvem	É a hospedagem das telas e de pequenas funções de autenticação, distribuída para carregar rápido.
JWT + bcrypt	Segurança	bcrypt embaralha a senha de forma irreversível; JWT é o crachá temporário que prova quem você é em cada pedido ao servidor.

EXEMPLO DE DIÁLOGO REAL NO WHATSAPP

Usuário: oi

Controla.AI: menu de boas-vindas em três bolhas curtas, e *nunca* pede valor numa saudação

Usuário: recebo 4500 por mês

Controla.AI: registra a renda uma única vez e confirma; não pergunta tipo, recorrência nem dia de pagamento

Usuário: gastei 45 no almoço

Controla.AI: ☒ Registrado: R\$ 45,00 em Alimentação” + link do painel

Usuário: quanto gastei esse mês?

Controla.AI: soma real do banco, comparação com a renda e projeção até o fim do mês

Usuário: quero juntar 5000 em 6 meses

Controla.AI: cria a meta com valor e prazo separados e responde com o progresso inicial

Por que “gastei 45” é difícil

Para o computador, essa frase não tem estrutura: não diz categoria, não diz data e “45” pode ser valor, quantidade ou prazo. É exatamente esse trabalho de estruturar que a IA faz.

Por que separar valor de prazo

Em “juntar 5000 em 6 meses” há dois números com significados opostos. O parser de metas trata cada um em campo próprio para não criar uma meta de R\$ 6.

Na prática

O produto não é “um app de finanças com um robô colado”. O canal de conversa é a interface principal, e o painel web existe para ver o conjunto. Toda a arquitetura das próximas páginas foi desenhada em torno dessa escolha: um servidor sempre ligado (para o WhatsApp funcionar), um interpretador de linguagem natural no meio e um banco relacional rígido no fim, garantindo que conversa informal produza dado formal.



A arquitetura em camadas

Cada camada com uma responsabilidade só

Imagine um restaurante. O salão atende o cliente, o caixa registra o pedido, a cozinha prepara e o estoque guarda os ingredientes. Ninguém faz o trabalho do outro: o garçom não cozinha e o cozinheiro não cobra. Um sistema bem organizado funciona igual, e isso tem nome: **arquitetura em camadas**.

No Controla.AI existem seis camadas. A vantagem prática dessa separação aparece na manutenção: se amanhã o WhatsApp mudar de biblioteca, só a camada de integração muda; o interpretador de linguagem, as regras financeiras e o banco continuam intactos. E se surgir um novo canal (Telegram, por exemplo), basta plugá-lo na mesma camada de domínio, nada mais precisa ser reescrito.

Camada	Onde fica no código	Responsabilidade	O que ela NÃO faz
Apresentação	<code>frontend/src/pages</code>	Mostrar telas: login, dashboard, metas, chat, administração. Coleta o clique e exibe o resultado.	Não calcula saldo nem decide regra de negócio; só pede e mostra.
API REST	<code>backend/src/api-routes.ts</code> e <code>extended-routes.ts</code>	Receber pedidos HTTP, validar o formato com Zod, checar o crachá JWT e devolver JSON.	Não fala com a OpenAI direto nem monta SQL na mão.
Domínio financeiro	<code>backend/api/*</code> e <code>goals-service.ts</code>	As regras de verdade: o que é despesa, como categorizar, como calcular progresso de meta e projeção de saldo.	Não sabe se o pedido veio do WhatsApp ou do navegador, e é por isso que serve aos dois.
Integração WhatsApp	<code>backend/whatsapp/*</code>	Manter a conexão viva, receber e enviar mensagens, identificar o telefone e dividir respostas em bolhas.	Não interpreta o conteúdo da frase; entrega para o domínio.
Integração OpenAI	<code>backend/api/parser.ts</code> , <code>media-processor.ts</code>	Transformar linguagem humana (texto, áudio, imagem, PDF) em dado estruturado e auditar custo.	Não grava nada no banco por conta própria.
Persistência	<code>backend/src/db/*</code> + PostgreSQL	Guardar com integridade: chaves, tipos, restrições de unicidade e regras de exclusão.	Não contém regra de produto; contém regra de consistência.

Por que o domínio é compartilhado

O WhatsApp e o chat web chamam a mesma função `processAgentMessage`. Consequência prática: uma correção de regra corrige os dois canais no mesmo instante, sem risco de comportamento divergente.

Por que a persistência é rígida

Deixar o banco recusar dado inconsistente é mais seguro do que confiar que todo programador vai lembrar de validar. As chaves estrangeiras funcionam como uma última linha de defesa, mesmo contra erro humano.

Na prática

As camadas não são enfeite de diagrama: elas explicam por que o projeto aceita quatro formatos de entrada diferentes (texto, áudio, foto, PDF) sem multiplicar a lógica financeira por quatro. Tudo é convertido para texto, o texto é convertido em `FinancialIntent`, e daí para frente existe um caminho único até o banco.



O caminho que um dado percorre

Diagrama textual: da mensagem ao gráfico

O diagrama abaixo é a mesma ideia do desenho técnico do documento oficial, escrito em texto para caber numa página e ser lido em voz alta na apresentação. Leia de cima para baixo: em cima estão as portas por onde o dado entra, no meio quem processa e embaixo onde ele fica guardado.

CANAIS DE ENTRADA

WhatsApp (biblioteca Baileys) | Pannel React publicado na Vercel

↓ ambos falam com o mesmo servidor ↓

SERVIDOR FASTIFY (Railway, sempre ligado)

src/index.ts (boot, CORS, health) → src/auth.ts (JWT) → src/api-routes.ts (CRUD) → src/extended-routes.ts (IA, metas, KPIs)

↓

DOMÍNIO + INTELIGÊNCIA (backend/api)

financial-agent.ts → onboarding-agent | goal-agent | income-classifier → parser.ts (OpenAI) → transaction-service.ts | insights.ts

↓

PERSISTÊNCIA

Drizzle ORM → PostgreSQL na Railway (16 tabelas, 18 chaves estrangeiras)

+ arquivos de apoio: .baileys-session/ (credenciais do WhatsApp) e .controlaai/runtime.json (modelo de IA escolhido)

O MESMO CAMINHO, PASSO A PASSO

- O dado entra.** Uma mensagem chega pelo WhatsApp ou um clique acontece no painel. Nos dois casos o que trafega é texto simples ou JSON, nada de arquivo pesado atravessando o sistema.
- O sistema descobre quem é.** No WhatsApp, o telefone é convertido em identificador de usuário; no painel, o crachá JWT já carrega esse identificador. Sem essa etapa, nada avança: é a regra que impede um dado entrar sem dono.
- Mídia é convertida em texto.** Áudio passa pelo Whisper, imagem passa pelo modelo de visão, PDF passa pelo extrator. Depois desta etapa, todo mundo é texto, e o resto do sistema não precisa saber a origem.
- O agente decide o que é aquilo.** Antes de chamar a inteligência artificial, o agente verifica se é saudação, se há um fluxo de meta em andamento, se é informação de renda. Isso economiza chamadas pagas e evita respostas fora de contexto.
- A IA estrutura a frase.** O parser devolve um objeto `FinancialIntent`: intenção, tipo, valor, categoria, descrição. Se a chave da OpenAI não estiver configurada, entra um interpretador local por expressões regulares, o sistema não para.
- A regra de negócio age.** Se for transação, o serviço resolve a categoria e grava. Se for pergunta, o módulo de indicadores faz as somas no banco e devolve número com contexto.
- O banco registra com integridade.** O PostgreSQL confere as chaves: usuário existe? categoria existe? Se algo estiver inconsistente, ele recusa, e é essa recusa que mantém a base confiável ao longo do tempo.
- A resposta volta pelo mesmo canal.** No WhatsApp, dividida em bolhas curtas para parecer conversa humana. No painel, como JSON que os gráficos consomem.
- Tudo fica auditado.** A mensagem vai para `whatsapp_messages`, a chamada de IA vai para `ai_logs` com custo e tempo, e a ligação entre a frase e o gasto fica registrada na chave estrangeira nº 18.

Na prática

Existe um único trajeto para o dado, e ele é auditável em cada estação. Numa apresentação, é possível pegar qualquer valor do dashboard e voltar até a frase original que o gerou, passando pela chamada de IA que a interpretou, com custo e tempo de processamento registrados.



Estrutura de pastas, a raiz do projeto

Onde cada coisa mora e por quê

Imagine um armário de escritório bem organizado: gavetas rotuladas, nada solto em cima da mesa. A organização de pastas de um projeto serve ao mesmo propósito, qualquer pessoa da equipe (e a banca) deve conseguir adivinhar onde um arquivo está só pelo nome da pasta. O princípio adotado no projeto é explícito no documento oficial: **poucas pastas, arquivos únicos e comentados linha a linha em português**.

controlaai/
├─ TCC_DOCUMENTACAO.md
├─ Documentacao TCC/
│ ┌─ MDs Arquitetura e PNGs Banco de Dados/
│ │ ┌─ ARQUITETURA_BANCO_COMPLETA.md
│ │ └─ CONEXOES_BANCO_DADOS.md
│ │ └─ TCC_DOCUMENTACAO.md
│ └─ Pngs BD/
├─ png/
└─ *.pdf / *.html
├─ frontend/
├─ backend/
│ ┌─ src/
│ └─ api/
│ └─ whatsapp/
│ └─ drizzle/
│ └─ scripts/
│ └─ .baileys-session/
│ └─ .controlaai/
├─ railway.toml / deploy.ps1
└─ .cursor/rules/

← documento oficial único do TCC (fonte de verdade)

← tudo que é entrega acadêmica

← colunas, tipos, diagrama ER

← as 18 chaves estrangeiras

← cópia do documento oficial

← diagramas em imagem

← cache de imagens usado pelos geradores de PDF

← PDFs finais entregues à banca

← painel web (React) + funções serverless da Vercel

← servidor, inteligência artificial e WhatsApp

← núcleo: Fastify, autenticação, banco

← agente de IA, parser, indicadores

← integração Baileys

← arquivos SQL do modelo de banco

← utilitários de banco e geradores de documentação

← credenciais do WhatsApp (nunca versionar)

← runtime.json com o modelo de IA escolhido

← configuração de publicação

← regra que obriga atualizar a documentação

POR QUE CADA PASTA EXISTE

Pasta / arquivo	Para que serve	Por que está separado assim
TCC_DOCUMENTACAO.md	Documento oficial único: arquitetura, rotas, banco, fluxos e histórico de alterações.	Ter um só documento evita a praga de vários arquivos de arquitetura contando versões diferentes da mesma história.
Documentacao TCC/	Entregas acadêmicas: PDFs, diagramas em imagem e os documentos de modelagem.	Separa o que é entrega da faculdade do que é código de produção.
frontend/	Tudo que o navegador executa, mais as pequenas funções de autenticação na borda.	Publica na Vercel de forma independente: uma mudança visual não exige reiniciar o servidor.
backend/src/	O servidor em si: boot, autenticação, rotas de CRUD e definição do banco.	É o miolo estável, muda pouco e com cuidado.
backend/api/	O cérebro: agente conversacional, parser da OpenAI, indicadores e memória.	É a parte que evolui mais rápido; isolada, pode mudar sem abalar o servidor.
backend/whatsapp/	O telefone: conexão, mensagens, bolhas, deduplicação e keep-alive.	Depende de uma biblioteca externa volátil; concentrar aqui limita o estrago de qualquer mudança dela.
backend/drizzle/	Guarda os arquivos SQL que descrevem o modelo do banco ao longo do tempo.	Permite recriar o banco do zero na ordem correta, em qualquer ambiente.
backend/scripts/	Utilitários: checar conexão, exportar dados e gerar estes PDFs.	Scripts de apoio não devem poluir o código que roda em produção.
.baileys-session/	Credenciais da sessão do WhatsApp (creds.json) após o QR ser escaneado.	Fora do controle de versão por segurança: são credenciais reais do número.
.cursor/rules/	Regra de projeto que obriga atualizar a documentação a cada alteração.	Transforma disciplina de documentação em automação, não em boa intenção.

Na prática

A organização é rasa de propósito. Em vez de cinco níveis de subpastas com nomes abstratos, há três pastas de código com nomes concretos: **src** é o servidor, **api** é a inteligência, **whatsapp** é o canal. Qualquer pessoa acha o arquivo em segundos, inclusive quem abrir o projeto pela primeira vez na banca.



Dentro de backend/src e scripts

O núcleo do servidor, arquivo por arquivo

A pasta `backend/src` é o miolo do sistema. É ela que liga o servidor, protege as rotas, descreve o formato do banco e oferece utilitários que o resto do código usa. Em geral essa pasta muda pouco. Quando muda, o cuidado é maior, porque quase tudo depende dela.

Termos rápidos desta página: *Fastify* é o framework que recebe pedidos HTTP (como um balcão de atendimento). *CORS* é a regra que diz quais sites podem chamar a API. *JWT* é o crachá temporário do login. *bcrypt* é a técnica que embaralha a senha para ninguém guardar o texto puro. *CRUD* significa criar, ler, atualizar e apagar registros.

ARQUIVOS DE BACKEND/SRC (O QUE CADA UM FAZ NA PRÁTICA)

Arquivo	Explicação simples
<code>index.ts</code>	É o interruptor geral. Sobe o servidor, lê as configurações, libera o acesso do painel web, registra a rota de saúde (<code>/health</code>), carrega as rotas, garante que exista um administrador, inicia o WhatsApp e fica escutando a porta 3333.
<code>env.ts</code>	Confere se as senhas e endereços necessários estão configurados. Se faltar algo importante, o servidor para na subida, e não no meio do atendimento a um usuário.
<code>auth.ts</code>	Cuida de cadastro e login. Embaralha a senha (<code>bcrypt</code>), compara na entrada, emite o crachá JWT e oferece o filtro que protege as rotas privadas.
<code>mailer.ts</code>	Manda e-mail de recuperação de senha e o código da verificação em duas etapas.
<code>api-routes.ts</code>	As operações do dia a dia: listar e criar lançamentos, categorias, resumo do painel e orçamento do mês.
<code>extended-routes.ts</code>	As rotas extras: chat com a IA, indicadores, metas, importação de PDF, histórico do WhatsApp e telas de administração.
<code>goals-service.ts</code>	Calcula o progresso real de cada meta somando os lançamentos do período certo (mês, trimestre, ano ou a janela do prazo).
<code>db/index.ts</code>	Abre a conexão com o PostgreSQL (até 10 conexões ao mesmo tempo) e exporta o objeto <code>db</code> usado no projeto inteiro.
<code>db/schema.ts</code>	O espelho das 16 tabelas em TypeScript. Colunas, tipos e ligações entre tabelas ficam declarados aqui uma vez só.
<code>db/ensure-admin.ts</code>	Na subida do servidor, garante que exista um usuário administrador, para o painel de gestão não ficar sem acesso.
<code>utils/money.ts</code>	Entende dinheiro escrito do jeito humano: “5k”, “5 mil”, “R\$ 30,00”. Também formata em reais e monta a chave do mês.
<code>utils/phone.ts</code>	Padroniza telefone brasileiro (com e sem o nono dígito). Isso é o que permite achar a conta certa quando a mensagem chega pelo WhatsApp.
<code>MAPA-SISTEMA.ts</code>	Lista dos arquivos de aplicação comentados em português. Serve de apoio direto à documentação do TCC.

BACKEND/SCRIPTS: FERRAMENTAS DE APOIO (NÃO RODAM NO DIA A DIA DO USUÁRIO)

Tipo de script	Para que serve	Exemplo do que a equipe usa
Checagem de ambiente	Confirmar se o banco responde e se as variáveis estão ok antes de demonstrar o sistema.	Script de ping/conexão com o PostgreSQL.
Geração de documentação	Montar o HTML e o PDF deste documento a partir dos MDs oficiais e dos diagramas.	<code>generate-ORGANIZACAO-SISTEMA-html.ts</code> e o gerador de PDF.
Utilitários de dados	Exportar ou inspecionar dados quando a equipe precisa auditar um caso real.	Scripts pontuais de consulta e conferência.
Apoio à apresentação	Gerar material visual (diagramas, PDFs) sem misturar isso com o código que atende o usuário.	Os PDFs da pasta Documentacao TCC.

Na prática

Se alguém perguntar “onde está a regra do login?”, a resposta é `auth.ts`. Se perguntar “onde as tabelas estão descritas no código?”, a resposta é `db/schema.ts`. Se perguntar “como este PDF foi gerado?”, a resposta está em `backend/scripts`. Essa divisão deixa o projeto legível para a equipe e para a banca.

A

Dentro de backend/api, whatsapp e frontend

O cérebro, o telefone e a vitrine

Se `src` é o esqueleto, `api` é o cérebro: ela transforma frase solta em ação concreta. São 22 arquivos pequenos, cada um com uma responsabilidade nomeada, a alternativa seria um arquivo gigante impossível de revisar.

BACKEND/API, ORQUESTRAÇÃO, OPENAI, CONSULTAS E MEMÓRIA

Arquivo	Responsabilidade
<code>financial-agent.ts</code>	O maestro. Define a ordem de prioridade: saudação → pós-cadastro → renda → onboarding → meta → parser → consulta → fallback.
<code>onboarding-agent.ts</code>	Perfil de renda mensal. Salva uma única vez e nunca repete as perguntas de tipo, recorrência ou dia de pagamento.
<code>goal-agent.ts</code> · <code>goal-parser.ts</code>	Criação conversacional de metas e extração separada de valor, prazo em meses e data limite.
<code>income-classifier.ts</code>	Distingue renda mensal de ganho pontual. Quando fica ambíguo, pergunta "1 = renda ou 2 = ganho?".
<code>parser.ts</code> · <code>prompts.ts</code>	Converte texto em <code>FinancialIntent</code> via GPT, com validação Zod e fallback local por expressões regulares.
<code>openai-client.ts</code> · <code>runtime-config.ts</code>	Conexão única com a OpenAI, cálculo de custo e troca de modelo pelo painel administrativo em tempo de execução.
<code>media-processor.ts</code>	Whisper para áudio e pdf-parse para extratos, converte mídia em texto.
<code>insights.ts</code>	Responde perguntas com números reais: quanto gastei, posso gastar, maior despesa, comparação de meses, relatórios.
<code>transaction-service.ts</code> · <code>category-resolver.ts</code>	Grava a transação e escolhe a categoria por sinônimos e inferência (pizza → Alimentação, uber → Transporte).
<code>financial-memory.ts</code> · <code>user-context.ts</code>	Guarda o que foi aprendido e monta o contexto financeiro enviado à IA em cada chamada.
<code>conversation-context.ts</code> · <code>conversation-history.ts</code>	Fase da conversa em memória e histórico recente para não repetir a mesma resposta.
<code>message-text.ts</code> · <code>transaction-intent.ts</code>	Normaliza o texto que chega e detecta por regex se é saudação, gasto, receita ou pergunta.
<code>assistant-response.ts</code> · <code>app-links.ts</code>	Templates de resposta e links do painel, com rodapés usados apenas quando fazem sentido.
<code>logger.ts</code> · <code>index.ts</code>	Auditoria em <code>ai_logs</code> e ponto de entrada serverless sem o WhatsApp.

BACKEND/WHATSAPP, O TELEFONE DO SISTEMA

Arquivo	Responsabilidade
<code>client.ts</code>	Socket do Baileys: QR Code, reconexão e proteção contra replay (ignora mensagens com mais de 4 minutos e o histórico dos 20 segundos seguintes à conexão).
<code>message-handler.ts</code>	O pipeline principal da mensagem que chega: identifica o usuário, trata mídia, chama o agente e responde em bolhas.
<code>user-resolver.ts</code> · <code>jid-resolver.ts</code>	Converte telefone em identificador de usuário e resolve os formatos internos de endereço do WhatsApp (LID e PN).
<code>whatsapp-bubbles.ts</code>	Divide a resposta longa em várias mensagens curtas, deixando a conversa com cara de humano.
<code>inbound-reply-guard.ts</code> · <code>message-dedup.ts</code>	Garantem que o robô só fale quando alguém falou com ele e que a mesma mensagem nunca seja processada duas vezes.
<code>keep-alive.ts</code> · <code>baileys-log.ts</code> · <code>routes.ts</code> · <code>session-utils.ts</code>	Verificação de saúde a cada 30 minutos, buffer de 500 linhas de log, rotas administrativas e localização da pasta de sessão.

FRONTEND, A VITRINE

Área	Arquivos principais	Função
Boot	<code>main.tsx</code> , <code>App.tsx</code>	Inicia o React e define as rotas do painel.
Páginas	<code>Dashboard</code> , <code>Goals</code> , <code>AiChat</code> , <code>Settings</code> , <code>Login</code> , <code>Register</code> , <code>ForgotPassword</code> , <code>ResetPassword</code> , <code>WhatsApp</code> , <code>AiLogs</code> , <code>AdminLgpd</code> , <code>AdminSubscribers</code> , <code>AdminAuditLogs</code>	Uma tela por arquivo, inclusive as telas administrativas.
Biblioteca	<code>lib/api.ts</code> , <code>lib/auth.tsx</code> , <code>lib/routes.ts</code>	Cliente HTTP central, contexto de autenticação e mapa de rotas.
Borda (Vercel)	<code>middleware.ts</code> , <code>vercel.json</code> , <code>api/auth/*</code> , <code>api/backend-proxy</code> , <code>api/relay</code>	Reescritas, proxy para o servidor e funções de login, recuperação de senha e dois fatores.

Na prática

Nenhum arquivo faz duas coisas. Quando o parser erra uma categoria, sabe-se exatamente onde olhar: `category-resolver.ts`. Quando o robô responde duas vezes, o suspeito é `message-dedup.ts`. Essa previsibilidade é o que permite manter um sistema desse tamanho com uma equipe de três pessoas.

A

O servidor Fastify: como ele liga

src/index.ts, env.ts e o health check

Imagine abrir uma loja de manhã: destrancar a porta, acender as luzes, ligar a máquina de cartão, conferir se o caixa tem troco e só então atender o primeiro cliente. Ligar um servidor é a mesma sequência, e a ordem importa. Se as rotas fossem registradas antes de o banco estar conectado, o primeiro pedido falharia.

OS OITO PASSOS DO BOOT, NA ORDEM EXATA

- 1
- Carrega as variáveis de ambiente via `env.ts`. São os segredos e endereços: endereço do banco, chave do JWT, chave da OpenAI. Se faltar algo obrigatório, o servidor recusa subir, falhar cedo e com mensagem clara é melhor que falhar no meio de um atendimento.
- 2
- Lê o modelo de IA salvo em disco com `initRuntimeConfig()`. O administrador pode trocar o modelo pelo painel, e a escolha fica em `.controlaai/runtime.json`, sobrevive a reinicializações.
- 3
- Cria a aplicação Fastify e libera o CORS. Isso autoriza explicitamente o endereço do painel na Vercel e o localhost a chamar a API. Sem essa permissão, o navegador bloqueia por segurança.
- 4
- Registra `GET /health`. É uma rota que só responde “estou vivo”. A Railway a consulta de tempo em tempo; se parar de responder, a plataforma reinicia o serviço automaticamente.
- 5
- Pendura as rotas em quatro blocos: autenticação, CRUD principal, rotas estendidas (IA e metas) e rotas administrativas do WhatsApp.
- 6
- Garante o usuário administrador com `ensureAdminUser()`. Assim o painel administrativo nunca fica inacessível, mesmo em banco recém-criado.
- 7
- Inicia o WhatsApp com `initWhatsApp()`, que sobe o Baileys e o verificador de 30 minutos. Pode ser desligado pela variável `ENABLE_WHATSAPP`, útil em ambiente de teste.
- 8
- Escuta a porta definida em `PORT` (3333 por padrão) e passa a atender pedidos.

COMO O SERVIDOR FALA COM O BANCO, SRC/DB/INDEX.TS

Decisão técnica	O que significa na prática
Pool de até 10 conexões	Abrir conexão com banco é caro. O pool mantém algumas prontas e reaproveita, como um táxi que fica no ponto em vez de voltar à garagem entre corridas.
SSL automático	A comunicação com o banco na nuvem vai criptografada, detectando sozinho quando o provedor exige.
<code>prepare: false</code> em pooler	Ajuste necessário quando o provedor usa um intermediário de conexões, que não suporta consultas pré-compiladas.
Exporta um único <code>db</code>	Todo o projeto usa o mesmo objeto de acesso. Isso evita conexões esquecidas abertas e centraliza qualquer ajuste de configuração.

Por que o health check é obrigatório

É o sinal vital que a nuvem observa. Sem ele, uma queda silenciosa do servidor só apareceria quando um usuário reclamasse, o que, num sistema de WhatsApp, pode levar horas.

Por que validar variáveis no boot

Uma chave de IA ausente descoberta às 3h da manhã, durante o atendimento, custa muito mais caro do que descoberta no segundo zero da inicialização.

Na prática
O servidor é uma peça só, que liga em ordem previsível e avisa alto quando algo falta. Para a banca, o roteiro de demonstração é direto: abrir `/health` e mostrar a resposta positiva prova que a aplicação está viva na nuvem antes de qualquer outro teste.



Autenticação: como o sistema sabe quem é você

bcrypt, JWT, dois fatores e regras de acesso

Imagine a portaria de um prédio. Na primeira visita você se cadastra e recebe um crachá com validade. Nas visitas seguintes, mostra o crachá e entra, o porteiro não precisa conferir seus documentos de novo. É exatamente assim que funciona a autenticação do Controla.AI: **bcrypt** guarda sua identidade com segurança e **JWT** é o crachá temporário.

CADASTRO, LOGIN E PROTEÇÃO DE ROTAS

- 1 Cadastro.** Os dados chegam e são validados com Zod (formato de e-mail, tamanho de senha). A senha passa pelo bcrypt com 10 rounds e vira um embaralhado irreversível. Nasce a linha em `users` e, junto, a linha de preferências em `user_settings`. No fim, o crachá JWT é emitido.
- 2 Login.** O sistema busca pelo e-mail, pega o embaralhado guardado e usa `bcrypt.compare` para conferir. Note o detalhe: ele não “desembaralha” a senha, isso é impossível por definição. Ele embaralha o que foi digitado e compara os resultados.
- 3 Crachá JWT.** Assinado em HS256 com validade de 7 dias, ele carrega o identificador do usuário. O navegador guarda e envia em cada pedido no cabeçalho `Authorization: Bearer`.
- 4 Porteiro das rotas.** O `authPreHandler` roda antes de cada rota protegida: confere a assinatura do crachá, carrega o usuário e o disponibiliza para a rota. Se o crachá for falso ou vencido, a resposta é 401 e a rota nem executa.
- 5 Área administrativa.** Um segundo porteiro, o `adminPreHandler`, restringe ao administrador as telas de QR Code do WhatsApp, logs de IA, assinantes e LGPD.
- 6 Recuperação de senha.** O usuário pede, recebe um código temporário por e-mail (`mailer.ts`), e o código é de uso único, restrições de unicidade no banco impedem reaproveitamento.
- 7 Dois fatores.** Funções dedicadas ativam, desativam, reenviam e verificam o segundo fator, reforçando o login de quem quiser.

OS MECANISMOS DE SEGURANÇA E O QUE CADA UM PROTEGE

Mecanismo	Implementação	Contra o que protege
Senha embaralhada	bcrypt com 10 salt rounds	Vazamento do banco: nem com a tabela em mãos alguém descobre a senha original.
Sessão por crachá	JWT HS256 com validade de 7 dias	Sessão eterna. Crachá roubado expira sozinho.
Rotas protegidas	<code>authPreHandler</code> com Bearer obrigatório	Acesso direto à API sem estar autenticado.
Separação por dono	Toda consulta filtra por <code>user_id</code>	Um usuário ver dados de outro, o erro mais grave possível num app financeiro.
Área administrativa	<code>adminPreHandler</code> restrito ao administrador	Usuário comum acessar QR do WhatsApp, logs ou lista de assinantes.
Validação de entrada	Schemas Zod em todas as rotas	Dado malformado ou malicioso entrando no banco.
CORS explícito	Apenas o endereço do painel e localhost	Site de terceiros chamar a API usando o crachá do usuário.
Credenciais do WhatsApp	Arquivos locais fora do controle de versão	Sessão real do número vazar em repositório.
Consentimento e auditoria	Tabelas e campos de LGPD no banco	Falta de rastro sobre o que foi consentido e sobre quem acessou o quê.

Na prática

Segurança aqui não é um recurso isolado: é uma corrente em que cada elo cobre a falha do anterior. Mesmo que uma rota esqueça de filtrar por dono, a chave estrangeira e o filtro por `user_id` na consulta ainda barram o acesso indevido. E como as senhas nunca existem em texto puro no sistema, nem a equipe do projeto consegue ler a senha de um usuário.



As rotas da API: o balcão de atendimento

CRUD principal, rotas estendidas e administrativas

Uma rota é um endereço que o servidor sabe atender, como um guichê numerado. O painel web não “entra no banco”: ele vai ao guichê certo e pede. Essa indireção é o que permite trocar o banco de lugar, mudar uma regra de cálculo ou acrescentar validação sem que o navegador precise saber.

API-ROUTES.TS, O CRUD PRINCIPAL (PREFIXO /API)

Método	Rota	O que faz	Tabelas envolvidas
GET	/transactions	Lista os lançamentos do usuário autenticado, com filtros de período e ordenação.	transactions + categories
POST	/transactions	Cria um lançamento manual vindo do painel, com validação de valor, tipo e data.	transactions
GET	/categories	Devolve as categorias globais somadas às personalizadas daquele usuário.	categories
GET	/dashboard/summary	Totais do mês: entradas, saídas e saldo, a fonte dos cartões do topo do painel.	transactions + budgets
PUT	/budgets/:month	Cria ou atualiza o orçamento de um mês (renda esperada e teto de gastos).	budgets

EXTENDED-ROUTES.TS, INTELIGÊNCIA, METAS E ADMINISTRAÇÃO

Método	Rota	O que faz	Tabelas envolvidas
POST	/ai/chat	Recebe a mensagem do chat web e chama o mesmo agente usado pelo WhatsApp.	ai_conversations, ai_logs, transactions
GET	/ai/welcome	Devolve boas-vindas ou inicia o onboarding, se o usuário for novo.	user_settings, budgets
GET	/ai/kpis	Indicadores financeiros calculados no banco para os cartões do dashboard.	transactions, budgets
GET	/ai/insights	Observações automáticas sobre o comportamento de gasto do mês.	transactions, categories
CRUD	/goals	Criar, listar, atualizar e arquivar metas, já com o progresso calculado.	goals, goal_checkpoints, transactions
POST	/imports/pdf	Recebe o extrato em PDF, registra a importação e processa os lançamentos.	document_imports, transactions
GET	/whatsapp/conversations	Histórico das mensagens de WhatsApp do usuário autenticado.	whatsapp_messages
Admin	/admin/ai/*	Logs de IA com custo e troca do modelo em tempo de execução.	ai_logs
Admin	/api/admin/whatsapp/*	Status, conectar, desconectar e ver logs do robô.	whatsapp_connection

O CICLO DE VIDA DE UM PEDIDO, DO CLIQUE À RESPOSTA

- 1 O navegador chama.** O `lib/api.ts` monta o pedido e anexa automaticamente o crachá JWT, nenhuma tela precisa lembrar de fazer isso.
- 2 A borda encaminha.** Na Vercel, a reescrita manda `/api/*` para o proxy, que repassa ao servidor na Railway.
- 3 O porteiro confere.** O `authPreHandler` valida o crachá e carrega o usuário no contexto do pedido.
- 4 O formato é validado.** O schema Zod recusa dado torto antes de qualquer consulta, valor negativo onde não pode, data inválida, campo faltando.
- 5 A regra roda.** A rota chama o serviço de domínio apropriado (transações, metas, indicadores), que é o mesmo usado pelo WhatsApp.
- 6 O banco responde.** O Drizzle gera o SQL, o PostgreSQL executa **sempre filtrando pelo dono** e devolve as linhas.
- 7 A resposta volta.** JSON limpo para a tela, ou texto em bolhas quando a origem é o WhatsApp. Se algo falhou, vem um código de erro claro em vez de página branca.

Na prática

A API é o único caminho para o dado, e todo caminho passa por três filtros obrigatórios: crachá válido, formato válido e dono correto. Para a banca, isso responde de uma vez a pergunta “como vocês garantem que um usuário não vê o dado do outro?": não é confiança na tela, é filtro no servidor mais chave estrangeira no banco.

A O agente de IA: quem decide o que fazer

financial-agent.ts e a ordem de prioridade

Imagine uma recepcionista experiente. Se alguém diz “bom dia”, ela cumprimenta, não pergunta o CPF. Se a pessoa estava preenchendo um formulário e volta com um número, ela entende que aquele número pertence ao formulário. Só quando a frase é realmente nova ela para e pensa. O `financial-agent.ts` é essa recepcionista, e a ordem em que ele verifica as coisas é o coração da experiência do produto.

Na prática, essa ordem existe por dois motivos. O primeiro é qualidade de conversa: nada é mais irritante que um robô pedindo valor quando você só disse “oi”. O segundo é custo: cada chamada à OpenAI é paga, então resolver com regra simples o que não precisa de IA é economia direta, visível em `ai_logs`.

AS NOVE ETAPAS VERIFICADAS EM ORDEM, ATÉ UMA RESPONDER

#	Verificação	Módulo	O que acontece
1	É saudação? (“oi”, “bom dia”)	message-text.ts	Menu de boas-vindas em três bolhas. Nunca pede valor numa saudação, e nem chama a IA.
2	Acabou de se cadastrar?	conversation-context.ts	Parabeniza e sugere criar a primeira meta, aproveitando o melhor momento de engajamento.
3	Há clarificação de renda pendente?	income-classifier.ts	Interpreta a resposta “1” como renda mensal e “2” como ganho pontual, usando o valor que ficou guardado na sessão.
4	É mensagem de renda, sem ser gasto?	income-classifier.ts	Decide entre salvar perfil mensal, registrar ganho isolado ou perguntar qual dos dois é.
5	Há onboarding em andamento?	onboarding-agent.ts	Continua o fluxo de renda, mas só se a pessoa ainda não tiver renda salva.
6	É só uma confirmação (“ok”, “beleza”)?	financial-agent.ts	Responde de forma leve e só pede renda se ela realmente estiver faltando.
7	É pedido ou fluxo de meta?	goal-agent.ts	Conduz valor → prazo → gravação da meta, com valor e prazo em campos separados.
8	Nada acima? Então interpreta.	parser.ts	Chama o GPT-4o-mini e recebe um FinancialIntent. Sem chave de IA, usa o interpretador local.
9	Ainda sem entender?	financial-agent.ts	Devolve o menu de boas-vindas em vez de um erro seco, o usuário nunca fica sem saída.

Regra da renda salva uma vez

Quando a pessoa informa a renda, o sistema grava o valor em `budgets` e preenche padrões silenciosos em `user_settings`. Na prática, ele nunca mais pergunta tipo, recorrência ou dia de pagamento, só reabre o assunto se o usuário pedir “configurar renda”.

Anti-repetição

Antes de enviar, o agente compara a resposta com as últimas mensagens já enviadas (lidas de `whatsapp_messages`). Se for igual, reformula. É o que evita a sensação de estar falando com um disco.

Mesmo agente nos dois canais

O WhatsApp e a rota `POST /ai/chat` chamam a mesma função. Consequência: não existe “o robô do Zap” e “o chat do site” com comportamentos diferentes.

Ambiguidade tratada, não ignorada

“5000” sozinho pode ser renda ou ganho. Em vez de adivinhar e errar, o agente pergunta uma única vez e guarda a resposta na sessão.

Na prática

O que parece “inteligência” é, em boa parte, ordem de decisão bem pensada. A IA entra quando é realmente necessária, interpretar linguagem livre, e fica de fora do que regra simples resolve melhor, mais rápido e de graça. É essa combinação que torna o assistente ao mesmo tempo agradável e economicamente viável.



O parser: de frase solta a dado estruturado

FinancialIntent, Whisper, visão, PDF e auditoria de custo

Para o computador, “gastei 45 no almoço” é só uma sequência de letras. Ele não sabe se 45 é valor, quantidade ou hora; não sabe que “almoço” sugere alimentação; não sabe se aconteceu hoje. O trabalho do **parser** é transformar essa frase num formulário preenchido, e esse formulário tem nome no projeto: `FinancialIntent`.

```
{
  "intent":    "transaction | query | report | goal | unknown",
  "type":     "expense | income | transfer",
  "value":    45.00,
  "category": "Alimentação",
  "description": "almoço",
  "date":    "2026-09-12",
  "queryType": "monthly_spending"
}
```

Esse objeto passa por uma validação com Zod antes de virar ação. Isso é importante: modelos de linguagem às vezes devolvem texto fora do formato pedido. Se o JSON não obedecer ao contrato, o sistema descarta e cai no plano B em vez de gravar lixo no banco.

CADA TIPO DE ENTRADA E O MODELO QUE A PROCESSA

Operação	Modelo	Entrada	Saída
parse	gpt-4o-mini	Texto + histórico de 10 mensagens + lista de categorias do usuário	<code>FinancialIntent</code> em JSON
transcribe	whisper-1	Áudio gravado no WhatsApp	Texto, que então segue para o parse
vision	gpt-4o-mini	Foto de nota fiscal ou comprovante	Texto com valor e estabelecimento
document	gpt-4o-mini	Texto extraído de extrato em PDF	Lista de lançamentos
chat	gpt-4o-mini	Conversa de vários turnos no painel	Resposta em linguagem natural

POR QUE ENVIAR CONTEXTO JUNTAMENTE COM A FRASE

- As categorias do usuário vão no pedido.** Assim o modelo escolhe entre as etiquetas que existem de fato, em vez de inventar “Refeições” quando a base usa “Alimentação”.
- As últimas 10 mensagens vão no pedido.** É o que permite entender “e no mês passado?”, sozinha, essa frase não tem sentido algum.
- O contexto financeiro vai no pedido.** Renda registrada, categorias mais usadas e fase da conversa, montados por `user-context.ts`.
- Existe plano B sempre.** Sem chave da OpenAI ou com a API fora, o `parseLocalIntent` usa expressões regulares para achar valor e palavras-chave. Menos preciso, mas o sistema continua registrando gastos.
- Toda chamada é auditada.** O `logger.ts` grava em `ai_logs`: operação, modelo, tokens de entrada e saída, custo em dólar, tempo em milissegundos e status.
- O modelo é trocável em produção.** O administrador escolhe pelo painel e a decisão fica em `.controlaai/runtime.json`, dá para testar um modelo mais barato sem reimplantar nada.

Por que numeric e não ponto flutuante

Dinheiro em tipo aproximado produz aquele clássico R\$ 0,01 de diferença no fechamento. A coluna de valor é `numeric(12,2)`, um tipo exato, o centavo nunca “evapora”.

Por que guardar o texto original

A coluna `raw_message` mantém a frase crua. Serve de prova para o usuário, de material para melhorar o parser e de rastro de auditoria para a banca.

Na prática

O parser é a fronteira entre o mundo informal do usuário e o mundo formal do banco relacional. De um lado, gente escrevendo “uns 45 no almoço”; do outro, uma coluna que só aceita número exato com duas casas e uma chave estrangeira que exige categoria existente. Toda a inteligência do projeto existe para atravessar essa fronteira sem perder informação nem inventar dado.



As consultas: perguntas viram somas no banco

Cada queryType explicado com a conta que ele faz

Quando o usuário pergunta “quanto gastei?”, o sistema não pede à inteligência artificial que *estime* um valor, isso seria um erro grave, porque modelos de linguagem inventam números com convicção. O papel da IA é apenas **classificar a pergunta** num tipo conhecido (o `queryType`). A partir daí, quem responde é o SQL, somando os dados reais.

OS SETE TIPOS DE CONSULTA IMPLEMENTADOS EM INSIGHTS.TS

queryType	Como o usuário pergunta	A conta que o sistema faz	Tabelas
monthly_spending	“Quanto gastei esse mês?”	Soma o valor de todas as transações de tipo despesa cuja data está dentro do mês corrente.	transactions
top_spending_days	“Quais dias eu gastei mais?”	Agrupar as despesas por dia, soma cada grupo e ordena do maior para o menor.	transactions
biggest_expense	“Qual foi minha maior despesa?”	Procura o maior valor entre as despesas do período e traz descrição e data.	transactions + categories
can_spend	“Posso gastar 500?”	Pega a renda registrada, subtrai o que já foi gasto, soma o que ainda vai entrar até o fim do mês e compara com o valor perguntado.	budgets + transactions + user_settings
health_check	“Como está minha situação?”	Monta um retrato: total de entradas, total de saídas, saldo, categorias que mais consomem e progresso das metas.	transactions + budgets + goals
month_comparison	“Gastei mais que no mês passado?”	Calcula dois períodos separadamente e apresenta a diferença em reais e em porcentagem.	transactions
income_profile_status	“Já cadastrei minha renda?”	Verifica se existe renda no orçamento do mês e se o onboarding foi concluído.	budgets + user_settings

EXEMPLO: A CONTA POR TRÁS DE “POSSE GASTAR 500?”

```
-- 1) Renda registrada para o mês corrente
SELECT total_income_expected FROM budgets
WHERE user_id = $1 AND month = to_char(NOW(), 'YYYY-MM');

-- 2) Quanto já foi gasto no mês
SELECT COALESCE(SUM(amount), 0) FROM transactions
WHERE user_id = $1 AND type = 'expense'
AND occurred_at >= date_trunc('month', NOW());

-- 3) Resposta = renda - gastos, cruzada com os dias restantes do mês
-- e com o limite de gastos definido em total_expense_limit
```

Relatórios por período

A função `generatePeriodReport` monta resumos semanais, mensais e anuais reaproveitando as mesmas somas, nada é recalculado de forma diferente em outro lugar.

Fonte única do dashboard

Os cartões “Ganhos”, “Gastos” e “Saldo” saem do mesmo cálculo consumido pelo assistente. Isso garante que o número do WhatsApp e o número da tela sejam sempre idênticos.

Por que a IA não calcula

Modelo de linguagem prevê a próxima palavra; ele não soma. Pedir cálculo a ele produziria respostas plausíveis e erradas, inaceitável em finanças.

O que a IA faz de fato

Entende variações infinitas da mesma pergunta (“quanto torrei”, “gastei muito?”, “tá caro esse mês?”) e mapeia todas para o mesmo `queryType`.

Na prática

A divisão de trabalho é explícita: a IA cuida da linguagem, o banco cuida dos números. Essa fronteira é o que permite afirmar à banca que nenhum valor apresentado ao usuário foi “gerado” por um modelo, todos vêm de consultas verificáveis sobre as linhas gravadas em `transactions`.



WhatsApp: como o sistema ganha um telefone

Baileys, QR Code, sessão e por que não pode ser serverless

Imagine contratar um funcionário só para atender o WhatsApp da empresa. Ele precisa de um telefone, precisa estar sempre por perto e precisa continuar logado mesmo depois do almoço. A biblioteca **Baileys** é esse funcionário: ela se conecta a um número usando o mesmo QR Code do WhatsApp Web e mantém a conversa aberta.

Na prática existe uma consequência arquitetural importante nessa escolha. Muitos sistemas modernos rodam em funções que ligam ao receber um pedido e desligam em seguida, é barato e escala bem. O WhatsApp não aceita esse modelo: a conexão é permanente e, se cair, a sessão se perde. Por isso o backend do Controla.AI roda na **Railway**, num processo que fica **sempre ligado**, e não em funções sob demanda.

COMO O ADMINISTRADOR CONECTA O NÚMERO, PASSO A PASSO

- 1 Login de administrador.** Apenas a conta administradora acessa a área de conexão, é uma credencial real de WhatsApp em jogo.
- 2 Abre /admin/whatsapp.** A tela consulta `GET /api/admin/whatsapp/status`, que lê a linha única de `whatsapp_connection` e mostra o estado atual.
- 3 Clica em Conectar.** A rota `POST /api/admin/whatsapp/connect` pede ao Baileys que inicie a conexão; o QR Code chega e é gravado na coluna `qr_code`, com o status virando `qr`.
- 4 Escaneia com o celular.** As credenciais são salvas em `.baileys-session/creds.json`, com a marca `registered: true`. Em produção, essa pasta vive num volume persistente para sobreviver a reinicializações.
- 5 Status vira “connected”.** A linha `main` registra o número pareado e o horário da conexão. A partir daí o robô atende **todos** os usuários.
- 6 O keep-alive assume.** A cada 30 minutos o sistema confere a saúde do socket e reconecta se necessário, sem intervenção humana.

DECISÕES TÉCNICAS E OS PROBLEMAS QUE ELAS RESOLVEM

Decisão	Por que foi necessária
Processo sempre ligado na Railway	A conexão do WhatsApp é permanente. Em ambiente serverless ela cairia a cada desligamento e exigiria novo QR constantemente.
Sessão em volume persistente	Sem volume, cada nova implantação apagaria as credenciais e o administrador teria que escanear o QR outra vez.
Um único registro em <code>whatsapp_connection</code>	A chave primária fixa no valor <code>main</code> impede duas linhas concorrentes descrevendo o mesmo aparelho.
Tabela sem <code>user_id</code>	O número é institucional, do sistema, não de uma pessoa. Ligá-lo a um usuário seria modelar errado a realidade.
Proteção contra replay	Ao reconectar, o WhatsApp reenvia mensagens antigas. Sem filtro, o robô responderia conversas de ontem, o sistema ignora mensagens com mais de 4 minutos e o histórico dos 20 segundos iniciais.
Buffer de 500 linhas de log	Permite ao administrador diagnosticar uma queda pela tela, sem precisar de acesso ao servidor.

Na prática

A escolha do WhatsApp como canal principal definiu a infraestrutura do projeto. Não foi possível optar pela hospedagem mais moderna e barata: foi preciso um servidor permanente, com disco persistente e verificação automática de saúde. É um caso concreto de requisito de produto determinando decisão técnica, e vale dizer isso na apresentação.



O pipeline completo de uma mensagem

message-handler.ts: do "chegou" ao "respondeu"

Quando uma mensagem chega, ela percorre uma esteira com estações fixas. Cada estação tem uma função única e pode interromper o processo, se o telefone não estiver cadastrado, por exemplo, nada é gravado e o usuário recebe o link de registro. Essa esteira é o `message-handler.ts`.

AS ONZE ESTAÇÕES DA ESTEIRA

- 1 **Baileys avisa.** O evento `messages.upsert` dispara com a mensagem nova.
- 2 **Filtro de replay.** O `client.ts` descarta mensagens com mais de 4 minutos e tudo que chega nos 20 segundos seguintes à reconexão, é o histórico reenviado pelo WhatsApp, não conversa nova.
- 3 **Deduplicação.** O `message-dedup.ts` verifica se aquele identificador de mensagem já foi visto. Se sim, ignora: ninguém quer ser respondido duas vezes.
- 4 **Resolução do endereço.** O `jid-resolver.ts` extrai o telefone real dos formatos internos do WhatsApp (LID e PN), consultando o mapeamento salvo na sessão.
- 5 **Identificação do usuário.** O `user-resolver.ts` testa variações do número brasileiro, com e sem o nono dígito, e encontra a pessoa em `users`. **Sem cadastro, nada é registrado**, o robô responde com o link de registro e encerra.
- 6 **Registro da entrada.** A mensagem é gravada em `whatsapp_messages` com direção `inbound`, tipo, conteúdo e o identificador do WhatsApp.
- 7 **Tratamento de mídia.** Se for áudio, o Whisper transcreve; se for imagem, o modelo de visão lê; se for PDF, o extrator puxa o texto. Depois desta estação, tudo é texto.
- 8 **O agente decide.** O `financial-agent.ts` aplica a ordem de prioridade das nove etapas e produz a resposta, gravando o que for necessário no banco.
- 9 **Vínculo mensagem ↔ gasto.** Se nasceu uma transação, o identificador dela é gravado em `whatsapp_messages.transaction_id`, é a chave estrangeira nº 18, que fecha o ciclo e garante rastreabilidade.
- 10 **Resposta em bolhas.** O `whatsapp-bubbles.ts` divide o texto no separador `|||` e envia várias mensagens curtas, como uma pessoa escreveria. Cada envio também é registrado, com direção `outbound`.
- 11 **Guarda de saída.** O `inbound-reply-guard.ts` só autoriza envio dentro do processamento de uma mensagem real recebida. Isso torna impossível, por construção, o sistema mandar mensagem não solicitada.

Por que dividir em bolhas

Um parágrafo enorme no WhatsApp parece circular automática. Três mensagens curtas parecem conversa. É detalhe de experiência com efeito direto no uso contínuo.

Por que nada sem cadastro

Se o sistema criasse dado para qualquer número que escrevesse, o banco encheria de registros sem dono e sem consentimento, problema técnico e problema de LGPD ao mesmo tempo.

Na prática

Cada proteção da esteira nasceu de um problema real observado: resposta duplicada, resposta a mensagem antiga, mensagem enviada sem pedido, dado criado sem dono. Documentar essas guardas é mostrar maturidade de engenharia, o sistema não é apenas o caminho felizmente bem-sucedido, é também o tratamento de tudo que pode dar errado.

As proteções do canal e a manutenção da conexão

Deduplicação, guarda de envio, keep-alive e logs

Manter um robô de WhatsApp em pé é menos sobre receber mensagem e mais sobre sobreviver ao que dá errado: quedas de rede, reenvio de histórico, servidor reiniciando no meio de uma conversa. As proteções abaixo são a resposta do projeto a cada um desses cenários.

CADA GUARDA, O PROBLEMA REAL QUE RESOLVE E COMO

Guarda	Arquivo	Problema real	Como resolve
Anti-replay por tempo	<code>client.ts</code>	Ao reconectar, o WhatsApp reenviou 30 mensagens antigas e o robô respondeu todas.	Descarta mensagens com mais de 4 minutos e tudo que chega nos 20 segundos após a conexão.
Deduplicação	<code>message-dedup.ts</code>	A mesma mensagem chegou duas vezes e gerou dois gastos iguais.	Guarda o identificador que o WhatsApp atribui e ignora repetições.
Guarda de envio	<code>inbound-reply-guard.ts</code>	Risco de o sistema enviar mensagem sem ninguém ter falado, caminho curto para virar spam e perder o número.	Só permite envio dentro do processamento de uma mensagem recebida de fato.
Anti-repetição	<code>conversation-history.ts</code>	O assistente repetia a mesma frase e parecia quebrado.	Compara a resposta com as últimas enviadas, lidas do banco, e reformula se houver repetição.
Keep-alive	<code>keep-alive.ts</code>	A conexão caiu de madrugada e ninguém percebeu até o primeiro usuário reclamar.	A cada 30 minutos verifica o socket e reconecta usando as credenciais salvas.
Buffer de logs	<code>baileys-log.ts</code>	Diagnosticar queda exigia acesso ao terminal do servidor.	Mantém 500 linhas em memória, exibidas na tela administrativa.
Resolução de endereço	<code>jid-resolver.ts</code>	O WhatsApp passou a usar identificadores internos que não são o telefone.	Traduz LID e PN para telefone usando o mapeamento gravado na sessão.
Variantes de telefone	<code>user-resolver.ts</code>	O usuário cadastrou o número sem o nono dígito e não era reconhecido.	Testa as variações brasileiras até encontrar a pessoa correta.

COMO O KEEP-ALIVE DECIDE O QUE FAZER

A cada 30 minutos (configurável em `WHATSAPP_KEEPLIVE_INTERVAL_MS`)

1. Existe sessão pareada? Se não, não faz nada, não há o que manter.
2. O socket está vivo (o objeto de usuário existe)? Se sim, faz um refresh preventivo leve.
3. Está offline? Reconecta usando as credenciais de `.baileys-session/`.
4. Atualiza `whatsapp connection` com o novo status, horário e eventual mensagem de erro.

O que o administrador vê

Uma tela com semáforo de status, o QR quando necessário, o número pareado, o tempo em pé e as últimas 500 linhas de log, tudo sem abrir terminal.

O que o usuário comum vê

Nada disso. Para ele existe apenas um número de WhatsApp que responde rápido. Toda essa engenharia é invisível, e é assim que deve ser.

Na prática

A diferença entre uma demonstração de laboratório e um sistema que funciona por semanas está exatamente aqui. Sem deduplicação, o usuário registra gasto dobrado; sem anti-replay, o robô fala com o passado; sem keep-alive, o canal morre silenciosamente. Essas guardas são o que sustenta a afirmação de que o projeto está em produção, e não só em teste.



O painel web: as telas do sistema

Uma página por arquivo, com rotas protegidas

O painel web é a vitrine do sistema. Enquanto o WhatsApp é ótimo para registrar na hora, é na tela que se vê o conjunto: gráficos, comparações entre meses, barras de progresso de meta e a lista completa de lançamentos. A organização segue uma regra simples de achar qualquer coisa, **uma tela, um arquivo**.

AS ROTAS DO PAINEL E O QUE CADA TELA FAZ

Rota	Arquivo	O que o usuário faz ali	Acesso
/	Dashboard.tsx	Vê os cartões de ganhos, gastos e saldo, os gráficos por categoria, a evolução do mês e a lista de transações. É a tela mais completa do sistema.	Usuário
/login	Login.tsx	Entra com e-mail e senha; pode usar o segundo fator se ativado.	Público
/register	Register.tsx	Cria a conta e informa o telefone que será usado no WhatsApp.	Público
/forgot · /reset	ForgotPassword.tsx · ResetPassword.tsx	Pede o código por e-mail e define a nova senha.	Público
/goals	Goals.tsx	Cria e acompanha metas, com progresso calculado a partir das transações reais.	Usuário
/ai	AiChat.tsx	Conversa com o assistente pelo navegador, com histórico na barra lateral.	Usuário
/settings	Settings.tsx	Ajusta perfil, tema, alertas, perfil de renda, exportação em CSV e privacidade.	Usuário
/admin/login	AdminLogin (fluxo)	Entrada exclusiva da administração.	Admin
/admin/whatsapp	WhatsApp.tsx	Conecta o número por QR, vê status, logs e escolhe o modelo de IA.	Admin
/admin/ai-logs	AiLogs.tsx	Consulta as chamadas de IA com tokens, custo e tempo.	Admin
/admin/lgpd	AdminLgpd.tsx	Trata pedidos de privacidade e consentimentos.	Admin
/admin/subscribers	AdminSubscribers.tsx	Acompanha assinaturas e planos.	Admin
/admin/audit-logs	AdminAuditLogs.tsx	Auditoria de ações sensíveis no sistema.	Admin
*	NotFound.tsx	Página amigável para endereço inexistente.	Público

Componentes reaproveitados

Layout.tsx (moldura e navegação), RequireAdmin (bloqueio de rota), ChartPlotArea (área de gráfico) e AppErrorBoundary (impede que um erro de componente apague a tela inteira).

Hooks próprios

use-capabilities descobre o que o plano permite, use-mobile adapta o layout ao celular e use-toast mostra avisos discretos.

Adaptação ao celular

Navegação horizontal rolável e o chat ocupando a altura real da tela (100dvh), porque no celular a barra do navegador rouba espaço.

Documentação no código

Os arquivos de aplicação têm comentários em português e cabeçalho apontando para o documento oficial. A pasta components/ui é biblioteca de terceiros e fica fora dessa regra.

Na prática

O painel não é um segundo sistema: é outra forma de olhar o mesmo banco de dados. Um gasto registrado por áudio no WhatsApp aparece no gráfico da tela sem nenhuma sincronização, porque nunca houve duas bases para sincronizar.



Como o painel conversa com o servidor

lib/api.ts, contexto de autenticação, middleware e proxy

Existe um detalhe de arquitetura que costuma surpreender: o navegador do usuário **não fala direto** com o servidor na Railway. Ele fala com a própria Vercel, que repassa o pedido. Esse desvio de uma casa resolve três problemas de uma vez, bloqueio de origem cruzada pelo navegador, exposição do endereço interno do servidor e ausência de um ponto único para ajustar cabeçalhos.

O CAMINHO DE UM PEDIDO, CAMADA POR CAMADA

- 1 A tela chama o cliente único.** Nenhuma página faz requisição por conta própria; todas usam `lib/api.ts`, que centraliza endereço base, cabeçalhos e tratamento de erro.
- 2 O crachá é anexado automaticamente.** O contexto de `lib/auth.tsx` guarda o JWT e o cliente o insere em todo pedido. Nenhum desenvolvedor precisa lembrar disso, e por isso ninguém esquece.
- 3 O middleware da Vercel intercepta.** O `middleware.ts` roda na borda, perto do usuário, e aplica regras de redirecionamento e cabeçalhos antes de qualquer coisa mais lenta acontecer.
- 4 As reescritas decidem o destino.** O `vercel.json` define que algumas rotas são atendidas por funções locais (login, recuperação de senha, dois fatores) e todas as outras `/api/*` vão para o proxy.
- 5 O proxy repassa.** A função `api/backend-proxy` encaminha ao endereço configurado em `BACKEND_URL`, preservando método, corpo e cabeçalhos.
- 6 O servidor responde.** O Fastify na Railway valida o crachá, executa a regra, consulta o PostgreSQL e devolve JSON.
- 7 A tela atualiza.** O React recebe os dados e redesenha apenas o que mudou, por isso o dashboard parece instantâneo mesmo com muitos gráficos.

CADA PEÇA DA BORDA E SUA FUNÇÃO

Peça	Onde fica	Função
<code>lib/api.ts</code>	Frontend	Cliente HTTP único: endereço base, crachá, tratamento de erro e conversão de resposta.
<code>lib/auth.tsx</code>	Frontend	Contexto de autenticação: guarda o JWT, expõe usuário e funções de entrar e sair.
<code>lib/routes.ts</code>	Frontend	Mapa central de rotas, evitando endereço escrito à mão espalhado pelas telas.
<code>middleware.ts</code>	Vercel (borda)	Executa antes de tudo: redirecionamentos e cabeçalhos de segurança.
<code>vercel.json</code>	Vercel	Reescritas que decidem o que é função local e o que vai para o proxy.
<code>api/backend-proxy</code>	Vercel (função)	Encaminha os pedidos ao servidor na Railway sem expor o endereço interno ao navegador.
<code>api/relay</code>	Vercel (função)	Repasse de envio de e-mail, usado quando o servidor não pode falar direto com o provedor SMTP.
<code>VITE_API_URL</code>	Variável do build	Endereço da API no ambiente local; em desenvolvimento o Vite encaminha para a porta 3333.

Na prática

Do ponto de vista do navegador existe um só domínio: o do painel. O servidor na Railway fica atrás do proxy, invisível. Isso elimina a classe de problemas de origem cruzada, reduz a superfície exposta e cria um ponto único onde acrescentar cabeçalho, registro de acesso ou limite de requisições, sem tocar em nenhuma tela.



As funções de autenticação na borda

Login, recuperação de senha, dois fatores e relay

Algumas operações não precisam do servidor grande na Railway: são rápidas, acontecem antes do usuário estar autenticado e ganham em ficar perto de quem chama. Login e recuperação de senha são exatamente assim. Por isso o projeto tem **funções serverless** na Vercel, pequenos programas que ligam ao receber um pedido, respondem e desligam.

Na prática, isso traz duas vantagens. A primeira é velocidade: a função roda na borda, próxima geograficamente do usuário. A segunda é resiliência: se o servidor principal estiver reiniciando após uma implantação, o login continua funcionando, porque essas funções falam com o banco por conta própria.

AS FUNÇÕES E O QUE CADA UMA RESOLVE

Função	Rota	O que faz
api/auth/login.ts	POST /api/auth/login	Confere e-mail e senha com bcrypt direto no banco, emite o crachá JWT e informa se o segundo fator é necessário.
api/auth/me.ts	GET /api/auth/me	Valida o crachá e devolve os dados do usuário autenticado, é como a tela sabe quem está logado após um recarregamento.
api/auth/forgot.ts	POST /api/auth/forgot	Gera um código de uso único, grava com prazo de validade e dispara o e-mail de recuperação.
api/auth/reset.ts	POST /api/auth/reset	Confere o código, troca a senha por um novo embaralhado bcrypt e invalida o código usado.
api/auth/otp-shared.ts	-	Biblioteca comum de códigos temporários: geração, validade, tentativas e invalidação. Compartilhada para não duplicar regra sensível.
api/auth/ping.ts	GET /api/auth/ping	Verificação simples de que as funções estão no ar.
api/auth-2fa-enable.ts	POST	Ativa o segundo fator e envia o primeiro código de confirmação.
api/auth-2fa-verify.ts	POST	Confere o código do segundo fator e libera a sessão completa.
api/auth-2fa-resend.ts	POST	Reenvia o código respeitando um intervalo mínimo, para evitar abuso.
api/auth-2fa-disable.ts	POST	Desativa o segundo fator após confirmação de identidade.
api/user-settings.ts	GET/PUT	Lê e grava preferências em <code>user_settings</code> sem passar pelo servidor principal.
api/relay/*	interno	Repassa o envio de e-mail ao provedor quando o caminho direto está indisponível.
api/backend-proxy/*	/api/*	Encaminha todo o resto ao Fastify na Railway.

Por que dividir a autenticação

Login precisa estar sempre disponível, ser rápido e independer do ciclo de implantação do servidor principal. São exatamente as características em que função serverless ganha.

Como a segurança se mantém

As funções usam as mesmas regras: bcrypt para senha, JWT com a mesma chave, restrições de unicidade no banco para códigos de uso único. Não existe caminho mais fraco.

Na prática

O sistema é híbrido de propósito: um servidor permanente onde o estado é obrigatório (o WhatsApp) e funções sob demanda onde velocidade e disponibilidade importam mais (autenticação). Escolher a ferramenta pelo problema, em vez de padronizar tudo numa só, é uma decisão de arquitetura que vale explicar à banca.



Publicação na Railway: o servidor sempre ligado

Backend, WhatsApp, PostgreSQL e volume persistente

Imagine alugar uma sala comercial em vez de trabalhar de casa: a sala tem energia estável, endereço fixo e alguém que liga o gerador quando falta luz. A **Railway** é essa sala para o Controla.AI. Ela hospeda três coisas que precisam estar de pé 24 horas: o servidor Fastify, o robô do WhatsApp e o banco PostgreSQL.

O QUE RODA NA RAILWAY E POR QUÊ

Componente	Por que precisa estar sempre ligado
Servidor Fastify	Atende o painel web e é o único caminho para o banco. Ponto de entrada de tudo.
Robô do WhatsApp (Baileys)	A conexão é permanente. Se o processo desligar, a sessão cai e o número para de responder.
PostgreSQL	É a fonte única de verdade. Precisa estar disponível sempre que qualquer canal for ler ou gravar.
Volume persistente	Guarda <code>/data/.baileys-session</code> . Sem ele, cada implantação exigiria escanear o QR novamente.
Verificação de saúde	A plataforma consulta <code>/health</code> periodicamente e reinicia o serviço se parar de responder.

DO COMMIT AO AR: O CICLO DE IMPLANTAÇÃO

- O código é enviado ao repositório.** A Railway detecta a mudança na branch configurada.
- Build.** O TypeScript é compilado para JavaScript na pasta `dist`. Erro de tipo aqui interrompe a implantação, a validação estática funciona como um teste automático.
- Banco alinhado ao código.** Antes de publicar, a equipe confere se o PostgreSQL está com o mesmo modelo descrito em `schema.ts`.
- Start.** O processo sobe com `node dist/src/index.js` e executa os oito passos de boot descritos na página 10.
- Health check.** A plataforma confirma que `/health` responde antes de direcionar tráfego para a nova versão.
- WhatsApp reconecta.** Como as credenciais estão no volume, a sessão é retomada sem novo QR Code.
- Monitoramento contínuo.** Logs ficam disponíveis no painel da plataforma e, para o WhatsApp, também na tela administrativa do próprio sistema.

COMANDOS DE BANCO USADOS NO DIA A DIA

```
cd backend
npm run db:push      # aplica o schema Drizzle no banco
npm run db:seed      # cria as categorias padrão, se estiver vazio
npm run db:setup     # push + seed em sequência
npm run db:check     # testa a conexão e mostra o resultado
npm run db:export-tcc # exporta diagramas, snapshot e a arquitetura completa
npm run dev          # desenvolvimento local com recarga automática
```

Na prática

A escolha da Railway não foi por preferência de marca, mas por requisito: o WhatsApp exige processo permanente e disco persistente. Manter o banco na mesma plataforma do servidor tem um ganho adicional relevante, a latência entre aplicação e banco fica mínima, o que aparece diretamente no tempo de resposta do assistente.

A

Vercel e as variáveis de ambiente

Onde cada segredo mora e para que serve

Enquanto a Railway hospeda o que precisa estar sempre ligado, a **Vercel** hospeda o que precisa estar sempre rápido: as telas do painel e as funções de autenticação. Ela distribui os arquivos por servidores próximos do usuário, então a primeira tela abre em milissegundos.

DIVISÃO DE RESPONSABILIDADES ENTRE AS DUAS NUENS

Na Vercel

Painel React compilado, middleware da borda, funções de login, recuperação de senha, dois fatores, preferências e o proxy que encaminha o resto para a Railway.

Na Railway

Servidor Fastify sempre ligado, robô do WhatsApp com volume de sessão, PostgreSQL e o webhook de cobrança do Stripe.

VARIÁVEIS DE AMBIENTE, O QUE SÃO E ONDE FICAM

Variável de ambiente é um dado de configuração que **não** mora no código: endereço do banco, chaves secretas, endereços públicos. A razão é simples e importante: chave secreta em repositório é chave vazada. Cada plataforma guarda as suas em painel protegido.

Variável	Onde	Obrigatória	Para que serve
DATABASE_URL	Railway + funções Vercel	Sim	Endereço completo do PostgreSQL, com usuário, senha e host.
JWT_SECRET	Railway + Vercel	Sim	Chave que assina e valida os crachás. Precisa ser idêntica nos dois lados, senão um não reconhece o crachá do outro.
OPENAI_API_KEY	Railway	Recomendada	Acesso à OpenAI. Sem ela o sistema funciona com o interpretador local, de forma reduzida.
OPENAI_MODEL	Railway	Não	Modelo padrão; o valor usual é gpt-4o-mini.
BACKEND_URL	Vercel	Sim	Endereço do servidor na Railway, usado pelo proxy.
FRONTEND_URL	Railway	Sim em produção	Libera o CORS e monta os links que o assistente envia nas mensagens.
REGISTER_URL	Railway	Não	Link de cadastro enviado a quem escreve no WhatsApp sem ter conta.
PORT	Railway	Não	Porta do servidor; o padrão é 3333.
BAILEYS_SESSION_DIR	Railway	Não	Caminho da sessão do WhatsApp; em produção aponta para o volume persistente.
ENABLE_WHATSAPP	Railway	Não	Permite subir o servidor sem o robô, útil em ambiente de teste.
WHATSAPP_KEEPLIVE_INTERVAL_MS	Railway	Não	Intervalo da verificação de saúde; padrão 1.800.000 ms (30 minutos).
REDIS_URL	Railway	Não	Cache auxiliar, quando habilitado.
VITE_API_URL	Build do frontend	Não	Endereço da API em desenvolvimento local.
STRIPE_*	Railway	Não	Chaves de cobrança e validação do webhook de assinatura.
SMTP_* / RELAY_*	Railway + Vercel	Não	Envio de e-mails de recuperação de senha e códigos de verificação.

Na prática

A mesma base de código roda em três ambientes, a máquina do desenvolvedor, a nuvem de produção e a demonstração da banca, mudando apenas essas variáveis. E há um detalhe que costuma render pergunta: `JWT_SECRET` precisa ser exatamente igual na Vercel e na Railway, porque o crachá emitido pela função de login é validado depois pelo servidor.



Como validar que tudo está funcionando

Roteiro de verificação, local e em produção

Um sistema com vários componentes precisa de um roteiro objetivo de verificação. O que segue é a sequência usada pela equipe depois de cada implantação, e também o roteiro sugerido para a demonstração à banca, porque vai do mais baixo nível (o servidor está vivo?) ao mais alto (o gasto apareceu no gráfico?).

ROTEIRO DE VERIFICAÇÃO EM PRODUÇÃO

- 1 O servidor está vivo?** Abrir `/health` no endereço da Railway. Resposta positiva significa processo em pé e porta respondendo.
- 2 O banco está acessível?** Rodar `npm run db:check`. Se o servidor sobe mas o banco não responde, o sintoma seria erro só na primeira consulta, melhor descobrir agora.
- 3 O painel abre?** Acessar o endereço na Vercel e confirmar que a tela de login carrega com estilo e sem erro no console.
- 4 O login funciona?** Entrar com um usuário de teste. Isso valida, de uma vez, a função serverless, o banco e a assinatura do crachá.
- 5 O proxy encaminha?** Com o painel aberto, ver o dashboard carregar dados. Se carrega, o caminho navegador → Vercel → Railway → PostgreSQL está inteiro.
- 6 O WhatsApp está pareado?** Abrir `/admin/whatsapp` e confirmar o status conectado, com número e horário de conexão.
- 7 O ciclo completo funciona?** Enviar “gastei 10 teste” pelo WhatsApp, receber a confirmação e, em seguida, ver o valor aparecer no dashboard. Este é o teste que prova o sistema inteiro.
- 8 A IA está sendo auditada?** Abrir `/admin/ai-logs` e conferir que a chamada recém-feita aparece com tokens, custo e tempo.

AMBIENTE LOCAL, DO ZERO

```
# Backend
cd backend
npm install
npm run db:push      # cria as tabelas no banco configurado
npm run db:seed      # categorias padrão
npm run dev          # sobe em http://localhost:3333

# Frontend (em outro terminal)
cd frontend
npm install
npm run dev          # sobe em http://localhost:5173
```

SINTOMAS COMUNS E ONDE INVESTIGAR

Sintoma	Causa provável	Onde olhar
O painel abre, mas nenhum dado carrega	<code>BACKEND_URL</code> errado ou servidor fora do ar	<code>/health</code> e as variáveis na Vercel
Login devolve 401 com senha correta	<code>JWT_SECRET</code> diferente entre Vercel e Railway	Variáveis das duas plataformas
O WhatsApp não responde	Sessão caiu ou o processo reiniciou sem volume	Status em <code>/admin/whatsapp</code> e os logs do Baileys
A IA não interpreta frases	Chave da OpenAI ausente ou sem crédito	<code>ai_logs</code> com status de erro e a mensagem registrada
Gasto registrado sem categoria	Tema novo, ainda sem categoria correspondente	<code>category-resolver.ts</code> e a tabela <code>categories</code>
O robô responde duas vezes	Falha de deduplicação após reconexão	<code>message-dedup.ts</code> e <code>whatsapp_messages.whatsapp_message_id</code>
Erro ao subir o servidor	Variável obrigatória faltando	Mensagem do <code>env.ts</code> no log de inicialização

Na prática

Existe um caminho de diagnóstico do mais simples ao mais complexo, e cada sintoma tem um lugar definido para ser investigado. Essa previsibilidade é o que permite operar o sistema sem adivinhação, e é o que transforma uma demonstração em algo reproduzível, e não em sorte.



A ponte entre o código e o banco de dados

schema.ts, Drizzle ORM e os documentos de modelagem

Aqui termina a Parte A e começa a Parte B. Antes de abrir as tabelas, vale entender como o banco de dados é **criado**, porque ele não foi desenhado numa ferramenta visual e depois copiado à mão. Ele nasce do próprio código, e essa decisão evita a divergência clássica entre “o que o sistema acha que existe” e “o que existe de fato no banco”.

1. backend/src/db/schema.ts, as 16 tabelas declaradas em TypeScript, com colunas, tipos, enums e chaves estrangeiras
↓ drizzle-kit gera o SQL correspondente
2. backend/drizzle/*.sql, arquivos SQL que acompanham a evolução do modelo no banco
↓ aplicadas em ordem em qualquer ambiente
3. PostgreSQL na Railway, banco railway, com 16 tabelas e 18 chaves estrangeiras ativas
↓ exportado por npm run db:export-tcc
4. ARQUITETURA_BANCO_COMPLETA.md + CONEXOES_BANCO_DADOS.md + diagramas PNG, **as fontes deste documento**

Repare no sentido das setas: a documentação é o **último** passo, não o primeiro. Os documentos de modelagem foram gerados a partir do banco real em produção, consultando o catálogo interno do PostgreSQL. Na prática, cada coluna, tipo e chave apresentados nas páginas seguintes é o que existe no servidor, não uma intenção de projeto.

O QUE CADA CAMADA GARANTE

Camada	Arquivo ou lugar	O que garante
Declaração	db/schema.ts	Uma definição única. O código e o banco não podem discordar, porque um é gerado do outro.
Tipagem estática	TypeScript + Drizzle	Consulta a coluna inexistente quebra na escrita do código, não em produção. Renomear uma coluna acusa todos os pontos afetados.
Versionamento	drizzle/0000...0013.sql	Histórico reproduzível: a mesma sequência de passos leva qualquer ambiente ao mesmo esquema.
Integridade	PostgreSQL	Chaves, tipos e restrições recusam dado inconsistente, mesmo que o código erre.
Documentação	Os três MDs + PNGs	Retrato fiel do banco real, gerado por script e não escrito à mão.
Apresentação	Este PDF	Traduz o retrato técnico para linguagem acessível, com uma página por tabela.

16

tabelas

18

chaves estrangeiras

147

colunas

6

grupos

Por que não desenhar o banco à mão

Diagrama feito manualmente envelhece na primeira alteração de código. Gerando o diagrama a partir do banco, a documentação nunca mente.

O que vem a seguir

A página 26 ensina a ler chaves e regras de exclusão; a 27 mostra o mapa dos 6 grupos; as 28 e 29 detalham as 18 ligações uma a uma; das 33 à 48, uma página inteira por tabela.

Na prática

Quando a banca perguntar “esse diagrama corresponde ao banco de verdade?”, a resposta é objetiva: ele foi extraído do banco em produção por script, na data registrada no cabeçalho dos documentos de origem. A modelagem não é um anexo do projeto, ela é derivada dele automaticamente.

B Como ler chaves, ligações e regras de exclusão

PK, FK, 1:1, 1:N, 0:1, CASCADE e SET NULL sem jargão

Um banco de dados relacional é um conjunto de tabelas que se apontam. Para entender qualquer modelagem, inclusive a das próximas páginas, bastam sete símbolos. Esta página explica cada um com uma analogia do mundo real, seguindo exatamente a tabela “Como ler este documento” do arquivo oficial de conexões.

OS SETE SÍMBOLOS QUE VOCÊ PRECISA CONHECER

Símbolo	Nome técnico	O que significa	Analogia do mundo real
PK	Chave primária	A coluna que identifica cada linha de forma única. Não repete e não pode ficar vazia.	O número da sua carteira de identidade: existe um só por pessoa, e ninguém tem o mesmo.
FK	Chave estrangeira	Uma coluna que guarda a chave primária de outra tabela, criando a ligação entre as duas.	O campo “nome do titular” num boleto: ele se refere a uma pessoa que existe em outro cadastro.
1:1	Um para um	Cada linha de A se liga a no máximo uma linha de B.	Pessoa e CPF: cada pessoa tem um CPF, cada CPF pertence a uma pessoa.
1:N	Um para muitos	Uma linha de A se liga a várias linhas de B.	Uma mãe e seus filhos: ela pode ter vários, cada filho tem uma mãe biológica.
0:1	Zero ou um (opcional)	A ligação pode existir ou não, a coluna aceita ficar vazia.	Cônjuge: uma pessoa pode ter um ou nenhum, e o cadastro precisa aceitar os dois casos.
CASCADE	Exclusão em cascata	Ao apagar a linha “pai”, o banco apaga automaticamente as linhas “filhas”.	Fechar a conta no banco: as movimentações daquela conta somem com ela.
SET NULL	Anular a referência	Ao apagar o “pai”, a coluna de ligação do “filho” fica vazia, mas o filho continua existindo.	Demitir o gerente de uma loja: a loja continua aberta, só fica sem gerente até chegar outro.

A regra geral do Controla.AI, registrada no documento de conexões, é direta: quase tudo gira em torno de `users`. Apagar um usuário remove a maior parte dos dados dele. Das 18 ligações do banco, **13 nascem de `users.id`**, e é isso que faz dela a tabela central da modelagem.

Por que CASCADE na maior parte

Dado pessoal deve sair com a pessoa. Além de ser exigência da LGPD, evita o problema técnico das linhas órfãs: registros apontando para um dono que não existe mais.

Por que SET NULL em alguns casos

Duas exceções propositais. Em `whatsapp_messages`, o log técnico do canal precisa sobreviver (um número pode escrever antes de ter conta). Em `ai_logs`, o custo operacional continua contabilizado mesmo depois da conta sair.

Por que não apagar gasto ao apagar categoria

Categoria é etiqueta; valor é fato. Perder a etiqueta é aceitável, perder o registro de R\$ 45 seria corromper o histórico financeiro. Por isso as três ligações de categoria usam SET NULL.

Por que a ligação 0:1 existe

A maioria das mensagens de WhatsApp é conversa, não gasto. Se o vínculo com transação fosse obrigatório, seria impossível registrar um simples “bom dia”.

Na prática

As regras de exclusão não são detalhe de configuração: são decisões de projeto com consequência direta. Escolher entre CASCADE e SET NULL é escolher o que deve sobreviver ao desaparecimento de um dado, e, em cada uma das 18 ligações deste banco, essa escolha foi feita de forma consciente, como as próximas duas páginas demonstram.



As 16 tabelas organizadas em 6 grupos

users no centro, tudo em volta

Dezesseis tabelas parecem muita coisa quando listadas em ordem alfabética. Organizadas por assunto, o desenho fica simples: existe uma tabela no centro e cinco grupos temáticos ao redor. Este é o mesmo mapa rápido do documento oficial de conexões, redesenhado para leitura em página impressa.



OS SEIS GRUPOS, TABELA POR TABELA

Grupo	Tabelas	Para que serve	Nº de tabelas
Usuário	users · user_settings	Identidade, login, vínculo com o telefone, preferências e perfil de renda. É a base de tudo: sem usuário identificado, nenhum dado entra.	2
Núcleo financeiro	transactions · categories · budgets · recurring_transactions	O dinheiro em si: cada gasto e receita, suas etiquetas, o orçamento do mês e as contas fixas que se repetem.	4
Metas	goals · goal_checkpoints	As promessas financeiras (não passar de X, juntar Y) e a foto mensal do progresso de cada uma.	2
WhatsApp	whatsapp_messages · whatsapp_sessions · whatsapp_connection	O canal: histórico de mensagens, estado das conversas em andamento e o aparelho oficial do sistema.	3
Inteligência artificial	ai_conversations · ai_logs · financial_memory · document_imports	O chat do painel, a auditoria de custo de cada chamada, o que a IA aprendeu e os arquivos importados.	4
Assinatura	subscriptions	O espelho local do contrato que o Stripe controla: plano, status e validade do período pago.	1

2

Usuário

4

Financeiro

2

Metas

8

Zap · IA · Assinatura

Por que users é o centro

É a única forma de garantir isolamento entre pessoas. Cada tabela de dado pessoal carrega o identificador do dono, e toda consulta filtra por ele. Isso torna o vazamento entre contas um problema de modelagem, não de disciplina de programação.

Por que uma tabela fica isolada

whatsapp_connection descreve o número institucional, não uma pessoa. Ligá-la a um usuário seria modelar a realidade de forma errada, e criaria a dúvida absurda de “de quem é o robô”.

Na prática

O modelo tem uma forma de estrela: um centro e raios. Essa escolha traz duas consequências práticas. A boa: consultas por usuário são diretas e rápidas, e o isolamento é estrutural. A que exige cuidado: apagar um usuário aciona 13 ligações de uma vez, motivo pelo qual cada regra de exclusão precisou ser decidida com atenção, e não copiada.

B As 18 ligações, uma a uma (1 de 2 · ligações 1 a 9)

Cada chave estrangeira explicada em linguagem de leigo

Cada bloco abaixo é uma ligação real do PostgreSQL, tirada da lista oficial de `CONEXOES_BANCO_DADOS.md`. O título mostra origem → destino, cardinalidade e o que acontece ao apagar. O parágrafo explica **por que** a ligação existe e o que mudaria no produto se ela não existisse.

FK 1 · users.id → user_settings.user_id (1:1 · CASCADE)

Cada pessoa tem exatamente **uma** folha de preferências (tema, alertas, perfil de renda). É como a ficha de configurações do celular: existe uma só, e ela pertence a um dono. Se a conta é apagada, essa folha vai embora junto, não faz sentido guardar preferência de quem não existe mais.

FK 2 · users.id → transactions.user_id (1:N · CASCADE)

Uma pessoa tem **muitos** gastos e receitas. Cada linha de `transactions` carrega o dono. Isso é o que garante que o João nunca veja o extrato da Maria: toda consulta filtra por esse campo. Apagar a conta apaga o histórico financeiro dela (CASCADE).

FK 3 · users.id → categories.user_id (1:N · CASCADE)

Além das categorias que já vêm prontas (Alimentação, Transporte...), o usuário pode criar as dele. Quando ele cria, a categoria fica marcada com o nome do dono. Categorias globais têm esse campo **vazio**, por isso a coluna aceita nulo: nulo significa 'serve para todo mundo'.

FK 4 · users.id → goals.user_id (1:N · CASCADE)

As metas ('gastar no máximo R\$ 400 em delivery', 'juntar R\$ 5.000') são pessoais. Cada meta sabe de quem é. Uma pessoa pode ter várias metas ativas ao mesmo tempo, e apagar a conta apaga as metas.

FK 5 · users.id → budgets.user_id (1:N · CASCADE)

O orçamento é mês a mês: junho, julho, agosto... Cada linha guarda a renda esperada e o teto de gastos de um mês só. Por isso 'vários registros' para a mesma pessoa: é um por mês, e não vários para o mesmo mês.

FK 6 · users.id → recurring_transactions.user_id (1:N · CASCADE)

Contas fixas (aluguel, internet, salário) ficam cadastradas uma vez e o sistema sabe que elas voltam todo mês. Cada uma pertence a um usuário. É como deixar um lembrete recorrente na agenda, só que financeiro.

FK 7 · users.id → ai_conversations.user_id (1:N · CASCADE)

O chat com a IA dentro do painel web guarda o histórico da conversa. Cada conversa pertence a uma pessoa, e ela pode ter várias (uma por assunto/mês). Apagar a conta apaga as conversas, é dado pessoal (LGPD).

FK 8 · users.id → financial_memory.user_id (1:N · CASCADE)

É a 'memória' que a IA constrói sobre o usuário: categorias que ele mais usa, perfil de renda, jeito de escrever. Cada aprendizado é uma linha ligada ao dono. Isso deixa o assistente mais certo com o tempo, sem misturar o aprendizado de pessoas diferentes.

FK 9 · users.id → document_imports.user_id (1:N · CASCADE)

Quando alguém envia um extrato em PDF pelo painel, o sistema registra esse envio: nome do arquivo, status do processamento e quantos lançamentos saíram dele. Cada importação tem dono, e o dono pode importar quantos arquivos quiser.

Como usar esta página na arguição

As nove primeiras ligações nascem todas de `users.id`. Em conjunto, elas respondem à pergunta “de quem é este dado?”. Sem elas, o sistema não conseguiria isolar a conta do João da conta da Maria, e apagar uma conta deixaria órfãos espalhados pelo banco.

B As 18 ligações, uma a uma (2 de 2 · ligações 10 a 18)

Cada chave estrangeira explicada em linguagem de leigo

Cada bloco abaixo é uma ligação real do PostgreSQL, tirada da lista oficial de CONEXOES_BANCO_DADOS.md. O título mostra origem → destino, cardinalidade e o que acontece ao apagar. O parágrafo explica **por que** a ligação existe e o que mudaria no produto se ela não existisse.

FK 10 · users.id → whatsapp_messages.user_id (1:N · SET NULL)

Toda mensagem que entra ou sai no WhatsApp fica registrada. Aqui há uma diferença importante: o vínculo é **SET NULL**. Na prática, se a conta for apagada, a mensagem continua existindo mas fica 'sem dono'. Motivo: um número pode mandar mensagem *antes* de ter cadastro, e o log técnico do canal precisa sobreviver para auditoria.

FK 11 · users.id → whatsapp_sessions.user_id (1:N · CASCADE)

Guarda o 'estado da conversa' de cada pessoa no WhatsApp, por exemplo, que ela está no meio da criação de uma meta e falta informar o prazo. É memória de curto prazo em formato JSON, e morre com a conta.

FK 12 · users.id → subscriptions.user_id (1:N · CASCADE)

Espelha no banco a assinatura que o Stripe controla: plano, status e até quando está pago. São 'vários' porque o histórico de assinaturas anteriores pode ficar guardado, mesmo que só uma esteja ativa.

FK 13 · users.id → ai_logs.user_id (1:N · SET NULL)

Cada chamada à OpenAI vira um registro com tokens, custo em dólar e tempo de resposta. O vínculo é **SET NULL** de propósito: mesmo que a conta seja excluída, o custo continua contabilizado para a operação do sistema, só deixa de estar associado a uma pessoa.

FK 14 · categories.id → transactions.category_id (1:N · SET NULL)

É a etiqueta do gasto: 'Alimentação', 'Transporte'. Uma categoria classifica muitos gastos. Se a categoria for apagada, o gasto **não** desaparece, ele só fica sem etiqueta (SET NULL). Perder a etiqueta é chato; perder o valor de R\$ 45 seria grave.

FK 15 · categories.id → goals.category_id (1:N · SET NULL)

Uma meta pode ser focada em uma categoria ('no máximo R\$ 300 em Lazer') ou geral (categoria vazia). Por isso o campo aceita nulo. Apagando a categoria, a meta continua viva, só perde o foco temático.

FK 16 · categories.id → recurring_transactions.category_id (1:N · SET NULL)

A conta fixa também recebe etiqueta: aluguel → Moradia, internet → Serviços. Mesmo comportamento das outras duas: apagar a categoria não apaga a conta fixa, só tira a classificação.

FK 17 · goals.id → goal_checkpoints.goal_id (1:N · CASCADE)

Cada meta gera uma 'foto' por mês: quanto foi gasto, qual era o limite, qual a porcentagem e se os alertas de 80% e 100% já foram enviados. Uma meta tem muitas fotos. Apagando a meta, as fotos vão junto, sem a meta elas não significam nada.

FK 18 · transactions.id → whatsapp_messages.transaction_id (0:1 · SET NULL)

É a ligação que fecha o ciclo do produto: a mensagem 'gastei 45 no almoço' aponta para o lançamento que ela gerou. É **opcional** (0:1) porque a maioria das mensagens é conversa, não gasto. Assim é possível provar a origem de cada centavo: do texto cru até a linha no banco.

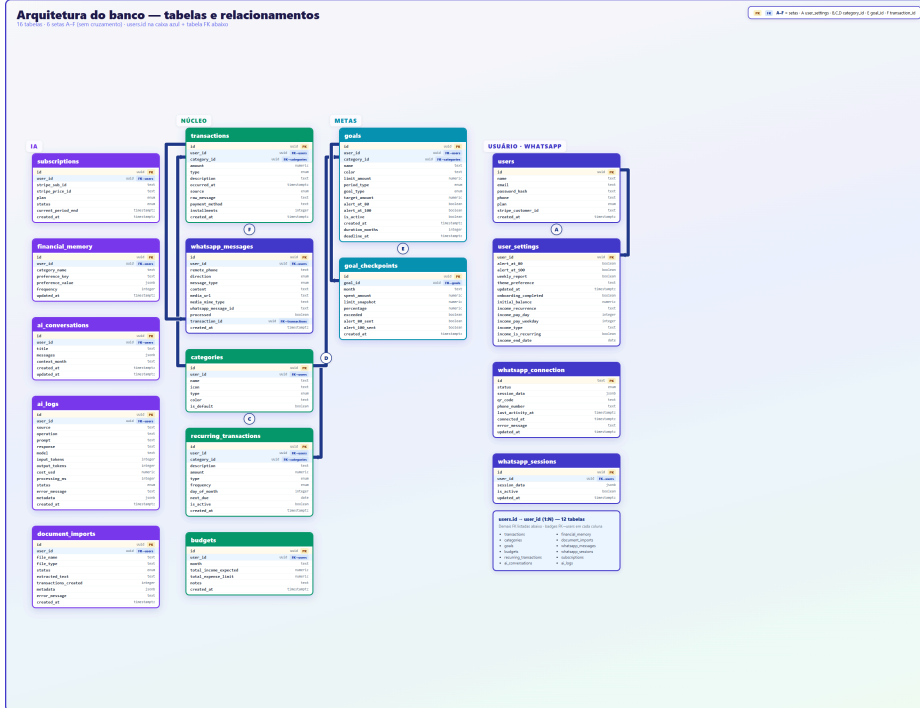
Como usar esta página na arguição

Aqui aparecem as duas políticas de exclusão misturadas: **CASCADE** (some junto) e **SET NULL** (fica sem dono). As FKs 14 a 16 protegem o histórico financeiro ao apagar categoria; a FK 18 fecha o rastro "mensagem do Zap → lançamento". As FKs 10 e 13 usam SET NULL de propósito: auditoria do canal e custo de IA sobrevivem à exclusão da conta.

Diagrama oficial de relacionamentos

Fonte: Pngs BD/arquitetura-banco-diagrama.png

A imagem abaixo é o diagrama oficial da modelagem, gerado a partir do banco em produção. Ele foi desenhado com linhas curtas entre tabelas vizinhas, em vez de linhas atravessando a página, justamente porque a versão anterior, com tudo ligado ao centro, ficava ilegível ao dar zoom. As caixas são as tabelas, as linhas são as 18 chaves estrangeiras e o destaque em `users.id` mostra o papel de centro dessa tabela.



Fonte: **ARQUITETURA_BANCO_COMPLETA.md** §4 · arquivo **arquitetura-banco-diagrama.png** · 2904 × 2272 px · 16 tabelas e 18 chaves estrangeiras extraídas do PostgreSQL em produção

Como ler a imagem

Cada caixa é uma tabela, com o nome no topo e as colunas listadas. As linhas ligam a chave primária de uma tabela à chave estrangeira de outra, sempre no sentido do dono para o dependente.

Onde está users

No papel de hub. Saem dela 13 das 18 ligações, o que visualmente explica por que ela é a primeira tabela detalhada neste documento.

A tabela solta

`whatsapp_connection` aparece sem nenhuma linha. Não é erro do diagrama: ela realmente não se liga a ninguém, porque descreve o aparelho do sistema.

LEGENDA, AS CAIXAS DO DIAGRAMA, POR BLOCO VISUAL

Bloco no diagrama	Caixas (tabelas)	O que esse bloco representa
Usuário / WhatsApp (direita)	users · user_settings · whatsapp_connection · whatsapp_sessions	A identidade e o canal. É onde nasce o identificador que todas as outras caixas guardam.
Núcleo (centro)	transactions · categories · budgets · recurring_transactions · whatsapp_messages	O dinheiro e suas etiquetas. A caixa de transações é a que recebe mais linhas, entra de usuário e de categoria, e sai para mensagens.
Metas (centro-direita)	goals · goal_checkpoints	As promessas financeiras e as fotos mensais de progresso. Note a linha única entre as duas: é a FK nº 17, em CASCADE.
IA (esquerda)	ai_conversations · ai_logs · financial_memory · document_imports	Chat do painel, auditoria de custo, aprendizado e importações. Todas ligadas apenas a usuário, nenhuma se liga entre si.
Assinatura (esquerda)	subscriptions	O espelho do contrato do Stripe, também ligado apenas a usuário.



Diagrama de conexões com detalhamento

Fonte: Pngs BD/arquitetura-banco-detalhes.png

Esta segunda imagem combina duas informações numa só página: o diagrama simplificado das ligações e a **tabela completa das 18 chaves estrangeiras**, com cardinalidade e regra de exclusão. É o material de consulta rápida durante a apresentação, o conteúdo das páginas 28 e 29 deste documento em forma visual.

Arquitetura completa — railway

Controla.AI · Sun Jun 07 2026 20:38:42 GMT-0300 (Horário Padrão de Brasília) · 16 tabelas · todas as colunas · 18 FK

Diagrama — linhas curtas entre tabelas vizinhas

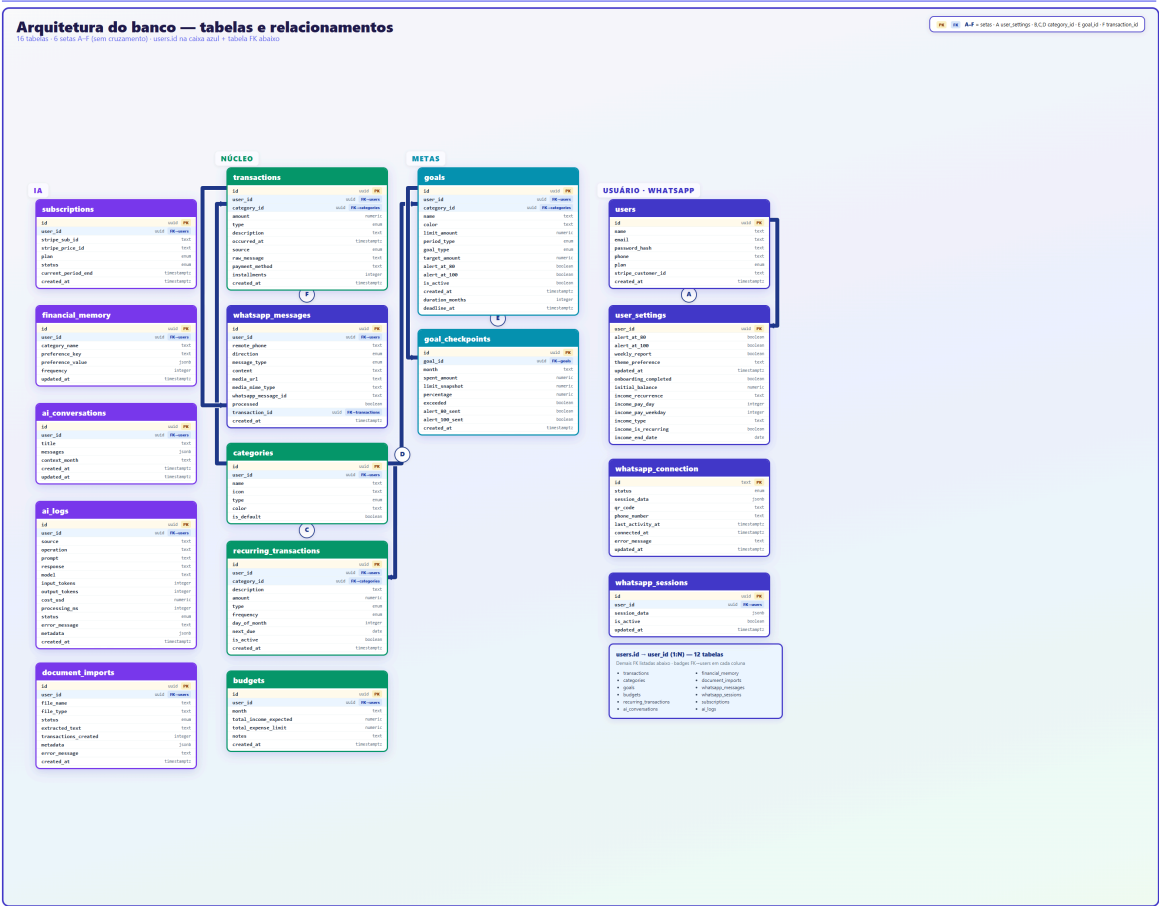


Tabela de relacionamentos (FK)

#	Origem (FK)	Destino (FK)	Card.	On delete
1	users.id	user_settings.user_id	1:1	CASCADE
2	users.id	transactions.user_id	1:N	CASCADE
3	users.id	categories.user_id	1:N	CASCADE
4	users.id	goals.user_id	1:N	CASCADE
5	users.id	budgets.user_id	1:N	CASCADE
6	users.id	recurring_transactions.user_id	1:N	CASCADE
7	users.id	ai_conversations.user_id	1:N	CASCADE
8	users.id	financial_memory.user_id	1:N	CASCADE
9	users.id	document_imports.user_id	1:N	CASCADE
10	users.id	whatsapp_messages.user_id	1:N	SET NULL
11	users.id	whatsapp_sessions.user_id	1:N	CASCADE
12	users.id	subscriptions.user_id	1:N	CASCADE
13	users.id	ai_logs.user_id	1:N	SET NULL
14	categories.id	transactions.category_id	1:N	SET NULL
15	categories.id	goals.category_id	1:N	SET NULL
16	categories.id	recurring_transactions.category_id	1:N	SET NULL
17	goals.id	goal_checkpoints.goal_id	1:N	CASCADE
18	transactions.id	whatsapp_messages.transaction_id	0:1	SET NULL

Fonte: ARQUITETURA_BANCO_COMPLETA.md \$4 · arquivo arquitetura-banco-detalhes.png · mesma modelagem, com cardinalidade e ON DELETE de cada ligação

Cardinalidade na imagem

1:1 aparece uma única vez (users e user_settings).
1:N é o padrão. 0:1 aparece só na ligação opcional entre transação e mensagem de WhatsApp.

Regras de exclusão

Onze ligações em CASCADE e duas em SET NULL a partir de users; as três de categoria também em SET NULL, protegendo o histórico financeiro.

Para que serve na banca

Responde de imediato perguntas do tipo “quantas chaves estrangeiras existem?” e “o que acontece se apagar um usuário?”, sem precisar navegar pelo documento.

B As decisões de modelagem e os tipos usados

UUID, numeric, JSONB, enums e integridade

Modelar um banco é escolher. Cada tipo de coluna e cada restrição resolve um problema e cria uma limitação. Esta página reúne as decisões tomadas no Controla.AI e o motivo de cada uma, é o tipo de justificativa que a banca costuma pedir.

ESCOLHAS DE TIPO E O QUE ELAS RESOLVEM

Escolha	Onde aparece	Por que foi escolhida
UUID como chave primária	15 das 16 tabelas	Identificador aleatório de 128 bits. Pode ser gerado pela aplicação antes de gravar, não revela quantidade de registros (ao contrário de um número sequencial) e nunca colide entre ambientes.
numeric(12,2) para dinheiro	amount, limit_amount, total_income_expected	Tipo decimal exato. Ponto flutuante produziria o clássico erro de centavo no fechamento do mês, inaceitável em finanças.
timestamptz para datas	Todas as colunas de data e hora	Guarda o fuso horário. Um gasto registrado às 23h em Curitiba não migra para o dia seguinte por causa de conversão.
enum para conjuntos fixos	type, source, status, direction, plan	O banco recusa valor fora da lista. Impossível gravar tipo "despeza" por erro de digitação.
JSONB para estrutura variável	messages, session_data, preference_value, metadata	Conteúdo cujo formato muda com frequência. Evita alterar a estrutura da tabela a cada novo campo, mantendo consultas eficientes.
text sem limite fixo	name, description, email	No PostgreSQL, text não é mais lento que varchar com tamanho. Limites arbitrários só criariam falhas futuras.
boolean para liga/desliga	is_active, processed, exceeded	Mais claro e mais leve que guardar 0 e 1 em texto ou número.
date sem hora	next_due, income_end_date	Vencimento é dia, não instante. Guardar hora aqui só abriria espaço para erro de fuso.

OS ENUMS DO POSTGRESQL E SEUS VALORES PERMITIDOS

Enum	Valores aceitos	Usado em
plan	free · pro · premium	users.plan · subscriptions.plan
category_type	expense · income	categories.type
transaction_type	expense · income	transactions.type · recurring_transactions.type
transaction_source	whatsapp · web · recurring · manual	transactions.source
goal_period	monthly · quarterly · yearly	goals.period_type
goal_kind	limit · saving	goals.goal_type
whatsapp_connection_status	disconnected · connecting · qr · connected · error	whatsapp_connection.status
whatsapp_message_direction	inbound · outbound	whatsapp_messages.direction
whatsapp_message_type	text · audio · image · document · video · other	whatsapp_messages.message_type
ai_log_status	success · error · pending	ai_logs.status
import_status	pending · processing · completed · failed	document_imports.status

Restrições de unicidade

E-mail único em `users` ; um orçamento por usuário e mês em `budgets` ; código de recuperação de senha de uso único. São regras impostas pelo banco, não pela tela.

Índices que importam

`whatsapp_messages` tem índice em `telefone`, `data` e `usuário`, as três formas de buscar conversa. Índice é o sumário do livro: evita ler tudo para achar uma linha.

Chave primária composta com FK

Em `user_settings`, o próprio `user_id` é a chave primária. É o que torna a relação 1:1 impossível de violar, mesmo por erro de código.

Singleton por chave fixa

Em `whatsapp_connection`, a chave primária é sempre o texto `main`. Duas linhas concorrentes descrevendo o robô deixam de ser possíveis.

Na prática

A integridade não depende de o programador lembrar de validar. Ela está declarada no banco: tipo exato para dinheiro, lista fechada para categoria de valor, chave estrangeira para vínculo e restrição de unicidade para o que não pode repetir. Se o código tentar gravar algo inconsistente, o PostgreSQL recusa, e é essa última linha de defesa que mantém a base confiável ao longo de meses de uso.

D users

Grupo Usuário · 8 colunas · Conta do sistema (login web + vínculo WhatsApp). Tabela central.

Imagine a ficha de matrícula de uma academia: uma folha por pessoa, com nome, documento e plano contratado. A tabela `users` é exatamente essa folha dentro do Controla.AI, e é a tabela mais importante de todas, porque 13 das 18 ligações do banco nascem dela.

Na prática, ela responde duas perguntas: **quem está entrando no painel web** (por e-mail e senha) e **quem está mandando mensagem no WhatsApp** (pelo telefone). Sem encontrar a pessoa aqui, o sistema não registra nada, é uma regra de segurança do projeto.

Quem escreve nesta tabela

`auth.ts` no cadastro web e `user-resolver.ts` quando um número novo fala no WhatsApp.

Quem lê esta tabela

Todas as rotas protegidas (o JWT carrega o id daqui) e todo o pipeline do agente de IA.

Se o registro pai for apagado

Apagar um usuário aciona 13 ligações: 11 em CASCADE (levam os dados) e 2 em SET NULL.

TODAS AS 8 COLUNAS, CÓPIA FIEL DE ARQUITETURA_BANCO_COMPLETA.MD §3, COM EXPLICAÇÃO

Coluna	Tipo	Aceita vazio?	Chave	O que guarda, em linguagem simples
<code>id</code>	<code>uuid</code>	não	PK	Código único gerado pelo banco. É o 'CPF interno' do sistema: todas as outras tabelas guardam esse número para dizer de quem é cada dado.
<code>name</code>	<code>text</code>	não	-	Nome exibido no painel e usado nas saudações do WhatsApp ('Oi, Davi!').
<code>email</code>	<code>text</code>	não	UNIQUE	Login do painel web. É único: duas contas não podem usar o mesmo e-mail.
<code>password_hash</code>	<code>text</code>	não	-	Senha embaralhada com <code>bcrypt</code> (10 rounds). O sistema nunca guarda a senha original, nem o administrador consegue ler.
<code>phone</code>	<code>text</code>	sim	-	Telefone no formato 55DDD9XXXXXXX. É a ponte com o WhatsApp: quem manda mensagem é localizado por aqui. Aceita vazio porque quem cadastra pelo site pode não informar.
<code>plan</code>	<code>enum plan</code>	não	-	<code>free</code> , <code>pro</code> ou <code>premium</code> . Controla o que a conta pode usar no produto.
<code>stripe_customer_id</code>	<code>text</code>	sim	-	Código do cliente no Stripe (a maquininha de cartão do sistema). Fica vazio para quem nunca assinou.
<code>created_at</code>	<code>timestamp tz</code>	não	-	Data e hora em que a conta nasceu, com fuso horário, importante para relatórios.

Entradas, quem aponta para cá

- Nenhuma. `users` é a raiz da modelagem: ninguém aponta para ela, ela é o destino de todos.

Saídas, para quem esta tabela aponta

- 13 ligações saem de `users.id`: `user_settings`, `transactions`, `categories`, `goals`, `budgets`, `recurring_transactions`, `ai_conversations`, `financial_memory`, `document_imports`, `whatsapp_messages`, `whatsapp_sessions`, `subscriptions` e `ai_logs`.
- 11 delas são CASCADE; `whatsapp_messages` e `ai_logs` são SET NULL.

```
-- Achar o dono de uma mensagem de WhatsApp pelo telefone
SELECT id, name, plan
FROM users
WHERE phone = '5541999999999';
```

Exemplo concreto de uso

Na prática, quando o número (41) 99999-9999 manda 'gastei 45 no almoço', o módulo `user-resolver.ts` testa variações do telefone (com e sem o nono dígito), acha essa linha, pega o `id` e só então o gasto pode ser gravado no nome da pessoa certa.



user_settings

Grupo Usuário · 14 colunas · Preferências e onboarding do usuário (1 registro por usuário).

Imagine a tela de 'Configurações' do seu celular: tema claro ou escuro, quais avisos você quer receber, seus dados de perfil. É uma tela só, e ela é sua. A tabela `user_settings` é essa tela guardada no banco, uma linha por pessoa, nunca duas.

Na prática ela tem duas funções. A primeira é visual e de comunicação: tema, relatório semanal e alertas de meta em 80% e 100%. A segunda é o **perfil de renda**: quanto a pessoa recebe, com que frequência e em que dia, é isso que permite ao assistente responder 'posso gastar 500?' com um número real.

Quem escreve nesta tabela

`auth.ts` cria a linha no registro; o `onboarding-agent` grava o perfil de renda; a tela `Settings` atualiza tema e alertas.

Quem lê esta tabela

O agente de IA (para não repetir perguntas), o dashboard (tema) e o serviço de alertas de meta.

Se o registro pai for apagado

CASCADE: apagar a conta apaga as preferências. Guardar preferência de quem não existe não faz sentido.

TODAS AS 14 COLUNAS, CÓPIA FIEL DE ARQUITETURA_BANCO_COMPLETA.MD §3, COM EXPLICAÇÃO

Coluna	Tipo	Aceita vazio?	Chave	O que guarda, em linguagem simples
<code>user_id</code>	<code>uuid</code>	não	PK/FK → <code>users.id</code>	Dono das configurações. Aqui ele é ao mesmo tempo chave primária e estrangeira, é esse truque que garante 'no máximo uma linha por pessoa' (relação 1:1).
<code>alert_at_80</code>	<code>boolean</code>	não	-	Se verdadeiro, avisa quando a meta chega a 80% do limite. É o 'pisca-alerta' antes do estouro.
<code>alert_at_100</code>	<code>boolean</code>	não	-	Se verdadeiro, avisa quando a meta estoura 100% do limite.
<code>weekly_report</code>	<code>boolean</code>	não	-	Liga ou desliga o resumo semanal de gastos.
<code>theme_preference</code>	<code>text</code>	não	-	Tema da interface (claro, escuro ou automático).
<code>updated_at</code>	<code>timestamptz</code>	não	-	Quando essas preferências mudaram pela última vez.
<code>onboarding_completed</code>	<code>boolean</code>	não	-	Marca se a conversa inicial de boas-vindas já terminou. É o que evita o assistente perguntar a renda outra vez.
<code>initial_balance</code>	<code>numeric</code>	sim	-	Saldo que a pessoa informou ter em conta no início. Serve de ponto de partida para a projeção de saldo.
<code>income_recurrence</code>	<code>text</code>	sim	-	Como a renda entra: <code>monthly_fixed</code> (todo mês igual), <code>weekly</code> ou <code>manual</code> . O padrão silencioso é <code>manual</code> .
<code>income_pay_day</code>	<code>integer</code>	sim	-	Dia do mês do pagamento (por exemplo, 5). Usado para projetar quanto ainda vai entrar.
<code>income_pay_weekday</code>	<code>integer</code>	sim	-	Dia da semana do pagamento, para quem recebe semanalmente.
<code>income_type</code>	<code>text</code>	sim	-	Origem da renda: salário, freelance, outro. Ajuda o assistente a falar a língua do usuário.
<code>income_is_recurring</code>	<code>boolean</code>	sim	-	Diz se a renda se repete todo período ou foi um ganho isolado.
<code>income_end_date</code>	<code>date</code>	sim	-	Data em que a renda deixa de valer (contrato por tempo determinado, por exemplo).

Entradas, quem aponta para cá

- `users.id` → `user_id`, cardinalidade 1:1, ON DELETE CASCADE.

Saídas, para quem esta tabela aponta

- Nenhuma FK de saída: esta tabela é uma 'folha' da modelagem.

```
-- Ler as preferências e o perfil de renda de uma pessoa
SELECT theme_preference, onboarding_completed, income_recurrence
FROM user_settings
WHERE user_id = $1;
```

Exemplo concreto de uso

Na prática, quando alguém diz no WhatsApp 'recebo 4500 por mês', o sistema grava o valor em `budgets` e, aqui, marca `onboarding_completed = true` com padrões silenciosos. Resultado prático: o assistente nunca mais pergunta a renda, a não ser que o usuário peça 'configurar renda'.

D transactions

Grupo Núcleo financeiro · 12 colunas · Lançamentos financeiros (gastos e receitas).

Imagine o extrato do seu banco: uma linha por movimento, com data, valor e descrição. A tabela `transactions` é o extrato do Controla.AI, e é o coração do produto. Se ela estiver correta, todos os gráficos e respostas da IA estarão corretos.

Na prática, é onde tudo termina: mensagem de WhatsApp, cadastro manual no painel, extrato em PDF importado ou conta fixa lançada automaticamente, tudo vira uma linha aqui. E é daqui que o dashboard tira 'Ganhos', 'Gastos' e 'Saldo', e que o assistente tira a resposta de 'quanto gastei?'.

Quem escreve nesta tabela

transaction-service.ts (via WhatsApp e chat), api-routes.ts (POST /transactions) e a importação de PDF.

Quem lê esta tabela

Dashboard, insights.ts, goals-service.ts (progresso de metas) e os relatórios por período.

Se o registro pai for apagado

CASCADE com users (o extrato morre com a conta) e SET NULL com categories (o valor fica, a etiqueta cai).

TODAS AS 12 COLUNAS, CÓPIA FIEL DE ARQUITETURA_BANCO_COMPLETA.MD §3, COM EXPLICAÇÃO

Coluna	Tipo	Aceita vazio?	Chave	O que guarda, em linguagem simples
<code>id</code>	uuid	não	PK	Identificador único do lançamento. É esse número que a mensagem de WhatsApp guarda para provar que foi ela quem gerou o gasto.
<code>user_id</code>	uuid	não	FK → <code>users.id</code>	Dono do lançamento. Obrigatório: não existe gasto sem dono no sistema.
<code>category_id</code>	uuid	sim	FK → <code>categories.id</code>	Etiqueta do lançamento (Alimentação, Transporte...). Aceita vazio porque o gasto pode entrar sem classificação e ser categorizado depois.
<code>amount</code>	numeric(12,2)	não	-	Valor em reais com duas casas. Usa tipo exato (não 'float') justamente para nunca arredondar centavo errado.
<code>type</code>	enum transaction_type	não	-	expense (saiu dinheiro) ou income (entrou dinheiro). É esse campo que separa a coluna de gastos da de ganhos no dashboard.
<code>description</code>	text	sim	-	O que foi, em palavras: 'almoço', 'uber para o trabalho'. Sai do parser ou do que o usuário digitou.
<code>occurred_at</code>	timestamptz	não	-	Quando o dinheiro se moveu de fato. Diferente de <code>created_at</code> : o usuário pode registrar hoje um gasto de ontem.
<code>source</code>	enum transaction_source	não	-	De onde veio: whatsapp, web, recurring ou manual. É a rastreabilidade do dado.
<code>raw_message</code>	text	sim	-	O texto original, cru, que gerou o lançamento ('gastei 45 no almoço'). Serve de prova e ajuda a melhorar o parser.
<code>payment_method</code>	text	sim	-	Forma de pagamento (pix, crédito, débito, dinheiro), quando a pessoa informa.
<code>installments</code>	integer	sim	-	Número de parcelas, para compras divididas no cartão.
<code>created_at</code>	timestamptz	não	-	Quando a linha foi criada no banco. Auditoria e ordenação técnica.

Entradas, quem aponta para cá

- `users.id` → `user_id` (1:N, CASCADE), o dono.
- `categories.id` → `category_id` (1:N, SET NULL), a etiqueta.

Saídas, para quem esta tabela aponta

- `transactions.id` → `whatsapp_messages.transaction_id` (0:1, SET NULL), é a FK nº 18, a que liga a mensagem ao gasto que ela criou.

```
-- Quanto a pessoa gastou no mês corrente
SELECT SUM(amount) AS total_gasto
FROM transactions
WHERE user_id = $1
AND type = 'expense'
AND occurred_at >= date_trunc('month', NOW());
```

Exemplo concreto de uso

Na prática, a frase 'gastei 45 no almoço' vira uma linha com `amount = 45.00`, `type = expense`, `source = whatsapp`, `raw_message` com o texto original e `category_id` apontando para Alimentação. Segundos depois, essa mesma linha é somada no cartão 'Gastos do mês' do painel web, mesma verdade, dois lugares.



categories

Grupo Núcleo financeiro · 7 colunas · Categorias de receita/despesa (padrão ou personalizadas).

Imagine as divisórias de uma pasta sanfonada: 'Mercado', 'Transporte', 'Lazer'. Cada gasto é um papel que você coloca em uma divisória. A tabela `categories` guarda essas divisórias, algumas já vêm de fábrica, outras o usuário cria.

Na prática ela é a base de todo relatório por assunto: o gráfico de pizza do dashboard, o 'onde gastei mais' do assistente e as metas por tema. É a única tabela que alimenta **três** ligações diferentes do banco.

Quem escreve nesta tabela

O seed inicial cria as padrão (is_default = true); category-resolver.ts cria personalizadas quando a IA encontra um tema novo.

Quem lê esta tabela

O parser (para escolher a etiqueta certa), o dashboard e as metas.

Se o registro pai for apagado

CASCADE com users (as personalizadas somem); mas quem aponta para ela usa SET NULL, nada financeiro é perdido.

TODAS AS 7 COLUNAS, CÓPIA FIEL DE ARQUITETURA_BANCO_COMPLETA.MD §3, COM EXPLICAÇÃO

Coluna	Tipo	Aceita vazio?	Chave	O que guarda, em linguagem simples
id	uuid	não	PK	Identificador da categoria, usado pelas três tabelas que a referenciam.
user_id	uuid	sim	FK → users.id	Dono da categoria. Vazio significa 'global' : a categoria vale para todos os usuários. É um detalhe elegante da modelagem, um campo nulo evita duplicar as categorias padrão para cada conta.
name	text	não	-	Nome exibido: 'Alimentação', 'Transporte', 'Assinaturas'.
icon	text	não	-	Nome do ícone mostrado na interface (por exemplo, utensils para comida).
type	enum category_type	não	-	expense ou income. Evita classificar um salário como 'Mercado'.
color	text	não	-	Cor em hexadecimal usada nos gráficos, para manter a mesma cor em todas as telas.
is_default	boolean	não	-	Marca as categorias que vêm de fábrica. As de fábrica não podem ser apagadas pelo usuário.

Entradas, quem aponta para cá

- users.id → user_id (1:N, CASCADE), só para as personalizadas.

Saídas, para quem esta tabela aponta

- categories.id → transactions.category_id (1:N, SET NULL)
- categories.id → goals.category_id (1:N, SET NULL)
- categories.id → recurring_transactions.category_id (1:N, SET NULL)

```
-- Categorias disponíveis para uma pessoa: as globais + as dela
SELECT name, icon, type
  FROM categories
 WHERE user_id IS NULL OR user_id = $1
 ORDER BY is_default DESC, name;
```

Exemplo concreto de uso

Na prática, quando alguém escreve 'pizza', o `category-resolver.ts` consulta esta tabela, encontra 'Alimentação' por lista de sinônimos e devolve o id. Se o tema não existir (por exemplo 'Pet'), ele cria uma categoria personalizada com o `user_id` preenchido, e ela passa a valer só para aquela pessoa.



budgets

Grupo Núcleo financeiro · 7 colunas · Orçamento mensal (renda esperada, limite de gastos).

Imagine um envelope por mês, com o quanto você espera receber escrito na frente e o quanto pretende no máximo gastar escrito atrás. A tabela budgets é a pilha desses envelopes: um por mês, por pessoa.

Na prática, é aqui que fica a **renda mensal** do usuário. E é essa renda que transforma perguntas subjetivas em respostas objetivas: 'posso gastar 500?' passa a ser renda menos gastos acumulados mais o que ainda vai entrar até o fim do mês.

Quem escreve nesta tabela

onboarding-agent.ts (saveIncomeProfileOnce) quando a pessoa informa a renda, e PUT /budgets/:month no painel.

Quem lê esta tabela

insights.ts (can_spend, health_check), o dashboard e o lembrete de perfil de renda.

Se o registro pai for apagado

CASCADE com users. Há restrição de unicidade por usuário + mês: não existem dois envelopes do mesmo mês.

TODAS AS 7 COLUNAS, CÓPIA FIEL DE ARQUITETURA_BANCO_COMPLETA.MD §3, COM EXPLICAÇÃO

Coluna	Tipo	Aceita vazio?	Chave	O que guarda, em linguagem simples
id	uuid	não	PK	Identificador do orçamento daquele mês.
user_id	uuid	não	FK → users.id	Dono do orçamento.
month	text	não	UNIQUE(user,month)	Mês no formato AAAA-MM (por exemplo 2026-09). Texto simples porque só interessa o mês, não o dia. Junto com user_id, forma uma chave única.
total_income_expected	numeric	sim	-	Renda esperada no mês. É o campo mais consultado da tabela: sem ele, o assistente não tem base para projetar saldo.
total_expense_limit	numeric	sim	-	Teto de gastos que a pessoa quer respeitar no mês.
notes	text	sim	-	Observações livres sobre o mês ('mês do IPVA', 'férias').
created_at	timestampz	não	-	Quando o envelope daquele mês foi criado.

Entradas, quem aponta para cá

- users.id → user_id (1:N, CASCADE).

Saídas, para quem esta tabela aponta

- Nenhuma FK de saída.

```
-- Renda esperada do mês atual
SELECT total_income_expected
FROM budgets
WHERE user_id = $1
AND month = to_char(NOW(), 'YYYY-MM');
```

Exemplo concreto de uso

Na prática, 'recebo 4500' cria (ou atualiza) a linha do mês 2026-09 com total_income_expected = 4500. Na pergunta seguinte, 'posso gastar 500 hoje?', o insights.ts soma os gastos já registrados, compara com esses 4500 e responde com número, não com 'depende'.

D

recurring_transactions

Grupo Núcleo financeiro · 11 colunas · Despesas/receitas fixas com vencimento recorrente.

Imagine os lembretes fixos da sua geladeira: 'aluguel dia 10', 'internet dia 15', 'salário dia 5'. Você escreve uma vez e eles valem para sempre. A tabela `recurring_transactions` é esse conjunto de lembretes, só que o sistema lê e lança sozinho.

Na prática, ela evita o trabalho repetitivo de cadastrar as mesmas contas todo mês e melhora a projeção de saldo: o sistema sabe o que **ainda vai** acontecer, não só o que já aconteceu.

Quem escreve nesta tabela

O painel web (cadastro de contas fixas) e o agente, quando o usuário diz que uma conta é mensal.

Quem lê esta tabela

O motor de lançamento automático (que cria transações com `source = recurring`) e a projeção do dashboard.

Se o registro pai for apagado

CASCADE com `users`; SET NULL com `categories`.

TODAS AS 11 COLUNAS, CÓPIA FIEL DE ARQUITETURA_BANCO_COMPLETA.MD §3, COM EXPLICAÇÃO

Coluna	Tipo	Aceita vazio?	Chave	O que guarda, em linguagem simples
<code>id</code>	<code>uuid</code>	não	PK	Identificador da conta fixa.
<code>user_id</code>	<code>uuid</code>	não	FK → <code>users.id</code>	Dono da conta fixa.
<code>category_id</code>	<code>uuid</code>	sim	FK → <code>categories.id</code>	Etiqueta da conta fixa (aluguel → Moradia). Opcional.
<code>description</code>	<code>text</code>	não	-	Nome da conta: 'Aluguel', 'Netflix', 'Salário'.
<code>amount</code>	<code>numeric</code>	não	-	Valor previsto em reais.
<code>type</code>	<code>enum</code> <code>transaction_type</code>	não	-	expense ou income, conta a pagar ou dinheiro a receber.
<code>frequency</code>	<code>enum</code>	não	-	Com que frequência repete: mensal, semanal, anual.
<code>day_of_month</code>	<code>integer</code>	não	-	Dia do mês do vencimento (por exemplo 10).
<code>next_due</code>	<code>date</code>	não	-	Próxima data de vencimento já calculada. É o campo que o sistema consulta para saber 'o que vence hoje'.
<code>is_active</code>	<code>boolean</code>	não	-	Liga/desliga a recorrência sem apagar o histórico. Cancelou a Netflix? Basta desativar.
<code>created_at</code>	<code>timestampz</code>	não	-	Quando a recorrência foi cadastrada.

Entradas, quem aponta para cá

- `users.id` → `user_id` (1:N, CASCADE).
- `categories.id` → `category_id` (1:N, SET NULL).

Saídas, para quem esta tabela aponta

- Nenhuma FK de saída.

```
-- Contas fixas que vencem hoje e ainda estão ativas
SELECT description, amount, type
FROM recurring_transactions
WHERE user_id = $1
AND is_active = true
AND next_due <= CURRENT_DATE;
```

Exemplo concreto de uso

Na prática, o aluguel de R\$ 1.200 com `day_of_month = 10` fica cadastrado uma única vez. Todo mês o sistema gera a transação correspondente (`source = recurring`) e empurra `next_due` para o mês seguinte, o usuário não digita nada e o dashboard já conta com esse valor.

goals

Grupo Metas · 15 colunas · Metas financeiras (teto de gasto ou poupança).

Imagine um cofrinho com um bilhete colado: 'juntar R\$ 5.000 até dezembro'. Ou um envelope com um aviso: 'no máximo R\$ 400 em delivery este mês'. A tabela `goals` guarda os dois tipos de promessa, a de guardar e a de não passar do limite.

Na prática, ela dá propósito ao controle financeiro. Sem meta, o app só mostra números; com meta, ele consegue avisar 'você já usou 80% do seu limite de Lazer' e mostrar barra de progresso no painel.

Quem escreve nesta tabela

goal-agent.ts (criação conversacional pelo WhatsApp) e o CRUD /goals do painel.

Quem lê esta tabela

goals-service.ts (calcula progresso somando transações), a página Goals e o serviço de alertas.

Se o registro pai for apagado

CASCADE com users e com goal_checkpoints; SET NULL com categories.

TODAS AS 15 COLUNAS, CÓPIA FIEL DE ARQUITETURA_BANCO_COMPLETA.MD \$3, COM EXPLICAÇÃO

Coluna	Tipo	Aceita vazio?	Chave	O que guarda, em linguagem simples
id	uuid	não	PK	Identificador da meta; é o número que cada checkpoint mensal guarda.
user_id	uuid	não	FK → <code>users.id</code>	Dono da meta.
category_id	uuid	sim	FK → <code>categories.id</code>	Categoria alvo, quando a meta é temática ('só Lazer'). Vazio = meta geral.
name	text	não	-	Nome da meta: 'Viagem para a praia', 'Segurar o delivery'.
color	text	não	-	Cor da meta na interface, para reconhecer de longe no painel.
limit_amount	numeric	não	-	Valor de referência: o teto quando é meta de limite, ou o valor base quando é poupança.
period_type	enum goal_period	não	-	Ciclo da meta de limite: monthly, quarterly ou yearly. Define quando o contador 'zera'.
goal_type	enum goal_kind	não	-	limit (não passar de X) ou saving (juntar X). Muda completamente a forma de calcular o progresso.
target_amount	numeric	sim	-	Alvo a alcançar nas metas de poupança.
alert_at_80	boolean	não	-	Avisar quando chegar a 80% do limite.
alert_at_100	boolean	não	-	Avisar quando estourar o limite.
is_active	boolean	não	-	Permite arquivar uma meta antiga sem perder o histórico de checkpoints.
created_at	timestamptz	não	-	Início da meta, nas metas de poupança, é o começo da janela de cálculo.
duration_months	integer	sim	-	Prazo em meses (5, 12...). Existe para separar valor de prazo e evitar confundir '5 meses' com R\$ 5.
deadline_at	timestamptz	sim	-	Data alvo já calculada a partir do prazo. Fecha a janela [created_at, deadline_at] usada no progresso.

Entradas, quem aponta para cá

- `users.id` → `user_id` (1:N, CASCADE).
- `categories.id` → `category_id` (1:N, SET NULL).

Saídas, para quem esta tabela aponta

- `goals.id` → `goal_checkpoints.goal_id` (1:N, CASCADE), o histórico mensal da meta.

```
-- Metas ativas de poupança com prazo definido
SELECT name, target_amount, duration_months, deadline_at
FROM goals
WHERE user_id = $1
AND is_active = true
AND goal_type = 'saving';
```

Exemplo concreto de uso

Na prática, 'quero juntar 5000 em 6 meses' vira uma linha com `goal_type = saving`, `target_amount = 5000`, `duration_months = 6` e `deadline_at` seis meses à frente. O `goals-service.ts` então soma as receitas dessa janela e devolve a porcentagem que aparece na barra de progresso.



goal_checkpoints

Grupo Metas · 10 colunas · Histórico mensal de progresso de cada meta.

Imagine que, no último dia de cada mês, alguém tira uma foto do seu cofrinho e anota atrás da foto: 'em setembro você gastou R\$ 320 dos R\$ 400 permitidos, 80%'. A tabela `goal_checkpoints` é o álbum dessas fotos.

Na prática, ela serve para duas coisas. Primeiro, contar a história: dá para mostrar num gráfico se a pessoa está melhorando mês a mês. Segundo, **não repetir alertas**: as colunas `alert_80_sent` e `alert_100_sent` registram que o aviso já foi enviado, para o usuário não receber a mesma mensagem cinco vezes.

Quem escreve nesta tabela

O serviço de metas, ao recalcular o progresso e ao disparar cada alerta.

Quem lê esta tabela

A página Goals (histórico), os relatórios de evolução e o verificador de alertas.

Se o registro pai for apagado

CASCADE com goals: sem a meta, o checkpoint perde qualquer sentido.

TODAS AS 10 COLUNAS, CÓPIA FIEL DE ARQUITETURA_BANCO_COMPLETA.MD §3, COM EXPLICAÇÃO

Coluna	Tipo	Aceita vazio?	Chave	O que guarda, em linguagem simples
<code>id</code>	<code>uuid</code>	não	PK	Identificador da 'foto' mensal.
<code>goal_id</code>	<code>uuid</code>	não	FK → <code>goals.id</code>	A qual meta esta foto pertence. É a única ligação da tabela.
<code>month</code>	<code>text</code>	não	-	Mês da foto no formato AAAA-MM.
<code>spent_amount</code>	<code>numeric</code>	não	-	Quanto foi efetivamente gasto (ou guardado) naquele mês.
<code>limit_snapshot</code>	<code>numeric</code>	não	-	Qual era o limite naquele momento . Guardar a cópia é essencial: se a pessoa mudar o limite depois, o histórico antigo continua verdadeiro.
<code>percentage</code>	<code>numeric</code>	não	-	Percentual atingido, já calculado, para o painel não ter que recalcular.
<code>exceeded</code>	<code>boolean</code>	não	-	Marca se o limite foi estourado naquele mês.
<code>alert_80_sent</code>	<code>boolean</code>	não	-	Registra que o aviso de 80% já saiu, evita mensagem repetida.
<code>alert_100_sent</code>	<code>boolean</code>	não	-	Registra que o aviso de estouro já saiu.
<code>created_at</code>	<code>timestampz</code>	não	-	Quando a foto foi tirada.

Entradas, quem aponta para cá

- `goals.id` → `goal_id` (1:N, CASCADE).

Saídas, para quem esta tabela aponta

- Nenhuma FK de saída.

```
-- Evolução de uma meta nos últimos meses
SELECT month, spent_amount, limit_snapshot, percentage, exceeded
FROM goal_checkpoints
WHERE goal_id = $1
ORDER BY month DESC;
```

Exemplo concreto de uso

Na prática, se em setembro a pessoa gastou R\$ 320 de um limite de R\$ 400, nasce uma linha com `percentage = 80` e `alert_80_sent = true`. Em outubro nasce outra linha, independente. Mesmo que em novembro ela aumente o limite para R\$ 600, a foto de setembro continua dizendo que o limite era R\$ 400, histórico honesto.

whatsapp_messages

Grupo WhatsApp · 12 colunas · Mensagens recebidas/enviadas pelo WhatsApp.

Imagine a transcrição completa de uma conversa de atendimento: tudo que o cliente falou e tudo que o atendente respondeu, na ordem. A tabela `whatsapp_messages` é essa transcrição, e é também a prova documental de como cada gasto entrou no sistema.

Na prática ela faz quatro trabalhos: guarda o histórico da conversa, evita que o robô responda duas vezes a mesma mensagem, alimenta a lógica de anti-repetição das respostas e, o mais importante, **liga a frase ao lançamento financeiro** pela coluna `transaction_id`.

Quem escreve nesta tabela

`message-handler.ts` (mensagens que chegam) e `whatsapp-bubbles.ts` (mensagens que saem).

Quem lê esta tabela

`conversation-history.ts` (últimas respostas para não repetir), o histórico do painel e a auditoria.

Se o registro pai for apagado

SET NULL com `users` e com `transactions`: o log do canal sobrevive, mesmo sem dono ou sem gasto.

TODAS AS 12 COLUNAS, CÓPIA FIEL DE ARQUITETURA_BANCO_COMPLETA.MD \$3, COM EXPLICAÇÃO

Coluna	Tipo	Aceita vazio?	Chave	O que guarda, em linguagem simples
<code>id</code>	<code>uuid</code>	não	PK	Identificador interno da mensagem no banco.
<code>user_id</code>	<code>uuid</code>	sim	FK → <code>users.id</code>	Dono da conversa. Aceita vazio porque um número pode escrever antes de ter cadastro, e essa tentativa também precisa ficar registrada.
<code>remote_phone</code>	<code>text</code>	não	-	Telefone do outro lado da conversa. Tem índice próprio, porque é a busca mais frequente.
<code>direction</code>	<code>enum</code>	não	-	<code>inbound</code> (chegou) ou <code>outbound</code> (o sistema enviou).
<code>message_type</code>	<code>enum</code>	não	-	<code>text</code> , <code>audio</code> , <code>image</code> , <code>document</code> , <code>video</code> ou <code>other</code> , define qual processador de mídia será usado.
<code>content</code>	<code>text</code>	sim	-	Texto da mensagem. No caso de áudio, recebe a transcrição feita pelo Whisper.
<code>media_url</code>	<code>text</code>	sim	-	Endereço do arquivo, quando a mensagem tem mídia.
<code>media_mime_type</code>	<code>text</code>	sim	-	Tipo do arquivo (<code>audio/ogg</code> , <code>image/jpeg</code> , <code>application/pdf</code>).
<code>whatsapp_message_id</code>	<code>text</code>	sim	-	Identificador que o próprio WhatsApp dá à mensagem. É a chave da deduplicação: se ele já apareceu, o sistema ignora o replay que o Baileys manda ao reconectar.
<code>processed</code>	<code>boolean</code>	não	-	Marca se a mensagem já passou pelo pipeline do agente. Evita processamento duplicado.
<code>transaction_id</code>	<code>uuid</code>	sim	FK → <code>transactions.id</code>	O lançamento que esta mensagem gerou, se gerou. É a FK nº 18, a que fecha o ciclo mensagem ↔ dinheiro.
<code>created_at</code>	<code>timestampz</code>	não	-	Momento da mensagem; tem índice para ordenar a conversa rapidamente.

Entradas, quem aponta para cá

- `users.id` → `user_id` (1:N, SET NULL).
- `transactions.id` → `transaction_id` (0:1, SET NULL).

Saídas, para quem esta tabela aponta

- Nenhuma FK de saída.

```
-- Mensagens que viraram gasto, com o valor ao lado
SELECT m.content, t.amount, t.occurred_at
FROM whatsapp_messages m
JOIN transactions t ON t.id = m.transaction_id
WHERE m.user_id = $1
ORDER BY m.created_at DESC;
```

Exemplo concreto de uso

Na prática, a banca pode pedir: 'mostre de onde veio esse R\$ 45'. O sistema faz um JOIN desta tabela com `transactions` e mostra a frase original, o horário e o telefone que enviou. Rastreabilidade completa, do texto humano até o número no gráfico.



whatsapp_sessions

Grupo WhatsApp · 5 colunas · Sessão conversacional por usuário (estado em JSON).

Imagine um atendente que anota num post-it 'esse cliente está no meio do cadastro da meta, falta o prazo'. Se ele sair para o almoço, outro atendente lê o post-it e continua de onde parou. A tabela `whatsapp_sessions` é esse post-it, guardado no banco em vez da memória do programa.

Na prática, é o que permite conversas de vários turnos. Sem ela, o assistente esqueceria o contexto a cada mensagem e perguntaria tudo de novo. Guardar no banco (e não só na memória) significa que, se o servidor reiniciar, a conversa não se perde.

Quem escreve nesta tabela

O pipeline do agente, ao abrir, atualizar ou encerrar um fluxo (onboarding, meta, clarificação de renda).

Quem lê esta tabela

financial-agent.ts, para descobrir se existe um fluxo em andamento antes de chamar o parser.

Se o registro pai for apagado

CASCADE com users: estado de conversa de conta apagada não tem utilidade.

TODAS AS 5 COLUNAS, CÓPIA FIEL DE ARQUITETURA_BANCO_COMPLETA.MD §3, COM EXPLICAÇÃO

Coluna	Tipo	Aceita vazio?	Chave	O que guarda, em linguagem simples
id	uuid	não	PK	Identificador da sessão.
user_id	uuid	não	FK → users.id	De quem é a sessão. Obrigatório: só existe contexto se existir pessoa.
session_data	jsonb	não	-	O estado em si, em formato JSON flexível: etapa atual, valores já coletados, última pergunta feita. Usar JSONB permite mudar o fluxo do agente sem precisar alterar o banco.
is_active	boolean	não	-	Diz se o fluxo está aberto. Ao terminar a criação da meta, vira falso.
updated_at	timestampz	não	-	Última alteração, serve para expirar sessões abandonadas.

Entradas, quem aponta para cá

- users.id → user_id (1:N, CASCADE).

Saídas, para quem esta tabela aponta

- Nenhuma FK de saída.

```
-- Sessão conversacional aberta de um usuário
SELECT session_data, updated_at
FROM whatsapp_sessions
WHERE user_id = $1
AND is_active = true;
```

Exemplo concreto de uso

Na prática, se a pessoa diz 'quero criar uma meta' e o robô pergunta o valor, o sistema grava aqui 'fluxo = meta, etapa = valor'. Quando ela responde '5000' dez minutos depois, o agente lê essa linha, entende que 5000 é o valor da meta, e não um gasto de R\$ 5.000.



whatsapp_connection

Grupo WhatsApp · 9 colunas · Estado global da conexão Baileys (singleton 'main'). Sem FK.

Imagine o aparelho de telefone da recepção de uma empresa: existe um só, ele não pertence a nenhum funcionário e todos usam. A tabela `whatsapp_connection` descreve esse aparelho: se está no gancho, se está chamando, e o QR Code que o administrador precisa escanear.

Na prática, é o painel de controle do robô. A tela `/admin/whatsapp` lê esta linha para mostrar 'conectado' ou 'aguardando QR'. É a **única tabela do banco sem ligação com users**, e isso é intencional, não esquecimento.

Quem escreve nesta tabela

`client.ts` (Baileys), a cada mudança de estado do socket, e `keep-alive.ts` a cada 30 minutos.

Quem lê esta tabela

As rotas administrativas `/api/admin/whatsapp/*` e a página de administração do WhatsApp.

Se o registro pai for apagado

Não se aplica: não há pai. É um registro único, criado uma vez e sempre atualizado.

TODAS AS 9 COLUNAS, CÓPIA FIEL DE ARQUITETURA_BANCO_COMPLETA.MD §3, COM EXPLICAÇÃO

Coluna	Tipo	Aceita vazio?	Chave	O que guarda, em linguagem simples
<code>id</code>	text	não	PK	Sempre o valor 'main'. É o truque de <i>singleton</i> : com a chave primária fixa, é impossível existirem duas linhas concorrentes descrevendo a conexão.
<code>status</code>	enum	não	-	<code>disconnected</code> , <code>connecting</code> , <code>qr</code> , <code>connected</code> ou <code>error</code> . É o semáforo que o administrador vê na tela.
<code>session_data</code>	jsonb	sim	-	Cópia técnica da sessão do Baileys, em JSON, para ajudar na recuperação da conexão.
<code>qr_code</code>	text	sim	-	O QR Code atual, em texto, que o painel transforma em imagem para o administrador escanear.
<code>phone_number</code>	text	sim	-	Número oficial que está pareado, o telefone do robô.
<code>last_activity_at</code>	timestampz	sim	-	Última vez que houve tráfego. É o que o keep-alive observa para decidir se precisa reconectar.
<code>connected_at</code>	timestampz	sim	-	Quando a conexão atual foi estabelecida. Serve para calcular o tempo em pé (uptime).
<code>error_message</code>	text	sim	-	Último erro em texto claro, para o administrador entender a queda sem abrir o log do servidor.
<code>updated_at</code>	timestampz	não	-	Última atualização da linha.

Entradas, quem aponta para cá

- Nenhuma, tabela isolada, conforme registrado em `CONEXOES_BANCO_DADOS.md`.

Saídas, para quem esta tabela aponta

- Nenhuma, ela não aponta para ninguém e ninguém aponta para ela.

```
-- Estado atual do robô de WhatsApp
SELECT status, phone_number, connected_at, error_message
FROM whatsapp_connection
WHERE id = 'main';
```

Exemplo concreto de uso

Na prática, quando o administrador clica em 'Conectar', o backend chama o Baileys, recebe o QR e grava `status = 'qr'` aqui. A tela consulta esta linha a cada poucos segundos e mostra o código. Depois do celular escanear, o status vira `'connected'` e o robô passa a atender todos os usuários, por isso não faz sentido essa tabela ter `user_id`.

 **ai_conversations**
Grupo IA · 7 colunas · Histórico do chat web com a IA.

Imagine o caderno de anotações de uma conversa com um consultor financeiro: cada caderno tem um título e, dentro, o diálogo completo na ordem. A tabela `ai_conversations` guarda esses cadernos do chat do painel web.

Na prática, ela dá memória ao chat da web: o usuário fecha o navegador, volta amanhã e a conversa está lá. É o equivalente, no painel, ao que `whatsapp_messages` faz no celular.

Quem escreve nesta tabela
extended-routes.ts, na rota POST /ai/chat, depois de cada resposta do agente.

Quem lê esta tabela
A página AiChat (lista lateral de conversas) e o parser, que usa o contexto recente para interpretar melhor.

Se o registro pai for apagado
CASCADE com users, é conteúdo pessoal e sai junto com a conta (LGPD).

TODAS AS 7 COLUNAS, CÓPIA FIEL DE ARQUITETURA_BANCO_COMPLETA.MD §3, COM EXPLICAÇÃO

Coluna	Tipo	Aceita vazio?	Chave	O que guarda, em linguagem simples
<code>id</code>	uuid	não	PK	Identificador da conversa.
<code>user_id</code>	uuid	não	FK → <code>users.id</code>	Dono da conversa.
<code>title</code>	text	sim	-	Título curto, geralmente gerado da primeira pergunta, para a pessoa reconhecer a conversa na lista.
<code>messages</code>	jsonb	não	-	O diálogo inteiro em JSON: uma lista de objetos com papel (usuário ou assistente), texto e horário. Guardar tudo em um campo evita uma tabela extra e é rápido de ler de uma vez.
<code>context_month</code>	text	sim	-	Mês de referência da conversa (AAAA-MM), para o assistente saber sobre qual período se está falando.
<code>created_at</code>	timestampz	não	-	Quando a conversa começou.
<code>updated_at</code>	timestampz	não	-	Última mensagem. É por este campo que a lista é ordenada.

Entradas, quem aponta para cá

- `users.id` → `user_id` (1:N, CASCADE).

Saídas, para quem esta tabela aponta

- Nenhuma FK de saída.

```
-- Conversas mais recentes do chat web
SELECT title, context_month, updated_at
FROM ai_conversations
WHERE user_id = $1
ORDER BY updated_at DESC
LIMIT 20;
```

Exemplo concreto de uso

Na prática, se o usuário pergunta no painel 'quanto gastei com transporte?' e depois 'e no mês passado?', a segunda pergunta só faz sentido por causa do histórico salvo aqui. O agente lê o JSON, entende que o assunto é transporte e responde certo.

 **ai_logs**

Grupo IA · 15 colunas · Log de chamadas OpenAI (parser, agente, áudio, visão).

Imagine a conta detalhada do telefone: cada ligação com duração, destino e preço. A tabela `ai_logs` é a conta detalhada da inteligência artificial: cada pedido feito à OpenAI, com quanto foi gasto em dólar, quanto tempo levou e se deu certo.

Na prática, resolve três problemas reais de um sistema com IA: **custo** (saber quanto o produto gasta por usuário), **desempenho** (achar as chamadas lentas) e **depuração** (ver exatamente qual texto foi enviado quando a resposta veio errada).

Quem escreve nesta tabela
logger.ts (logAiOperation), chamado por parser.ts, insights.ts e media-processor.ts.

Quem lê esta tabela
A tela administrativa /admin/ai-logs e as análises de custo do TCC.

Se o registro pai for apagado
SET NULL com users: o custo continua registrado para a operação mesmo depois da conta ser excluída.

TODAS AS 15 COLUNAS, CÓPIA FIEL DE ARQUITETURA_BANCO_COMPLETA.MD §3, COM EXPLICAÇÃO

Coluna	Tipo	Aceita vazio?	Chave	O que guarda, em linguagem simples
id	uuid	não	PK	Identificador do registro de log.
user_id	uuid	sim	FK → users.id	Quem originou a chamada. Vazio quando é chamada de sistema ou quando a conta foi excluída depois.
source	text	não	-	Canal de origem: whatsapp, web ou admin.
operation	text	não	-	Qual operação foi: parse, chat, transcribe, vision ou document.
prompt	text	sim	-	O texto enviado ao modelo. É o que permite reproduzir o problema depois.
response	text	sim	-	A resposta bruta recebida do modelo.
model	text	sim	-	Modelo usado (por exemplo gpt-4o-mini ou whisper-1). Guardar isso importa porque o administrador pode trocar o modelo em tempo de execução.
input_tokens	integer	sim	-	Quantidade de 'pedaços de texto' enviados. Tokens são a unidade de cobrança da OpenAI.
output_tokens	integer	sim	-	Quantidade de tokens gerados na resposta.
cost_usd	numeric	sim	-	Custo estimado da chamada em dólares, calculado a partir dos tokens e do preço do modelo.
processing_ms	integer	sim	-	Tempo total em milissegundos, usado para achar gargalos.
status	enum ai_log_status	não	-	success, error ou pending.
error_message	text	sim	-	Mensagem de erro, quando a chamada falha (chave inválida, limite excedido, tempo esgotado).
metadata	jsonb	sim	-	Informações extras em JSON, como o intent devolvido pelo parser. Campo livre para não precisar alterar a estrutura da tabela a cada novo detalhe.
created_at	timestampz	não	-	Momento da chamada.

Entradas, quem aponta para cá

- users.id → user_id (1:N, SET NULL).

Saídas, para quem esta tabela aponta

- Nenhuma FK de saída.

```
-- Custo da IA nos últimos 30 dias, por operação
SELECT operation, COUNT(*) AS chamadas, SUM(cost_usd) AS custo
  FROM ai_logs
 WHERE created_at > NOW() - INTERVAL '30 days'
 GROUP BY operation
 ORDER BY custo DESC;
```

Exemplo concreto de uso

Na prática, a equipe consegue afirmar, com dado e não com estimativa: 'interpretar uma mensagem custa cerca de US\$ 0,0002 e leva 900 ms'. Para o TCC, é a prova de que o uso de IA foi medido e é economicamente viável.



financial_memory

Grupo IA · 7 colunas · Preferências aprendidas pela IA (JSON por chave).

Imagine o garçom do restaurante que você frequenta: depois de algumas visitas, ele já sabe que você prefere mesa perto da janela e não pergunta mais. A tabela `financial_memory` é essa memória do assistente sobre cada usuário.

Na prática, ela é o que faz o app parecer inteligente com o tempo. Se a pessoa sempre classifica 'ifood' como Alimentação, esse aprendizado fica guardado aqui e nas próximas vezes a categoria é acertada de primeira, sem gastar chamada de IA.

Quem escreve nesta tabela

financial-memory.ts (setUserPreference), chamado após transações e após salvar o perfil de renda.

Quem lê esta tabela

O parser, o category-resolver e o user-context, que monta o contexto financeiro enviado à IA.

Se o registro pai for apagado

CASCADE com users: é dado pessoal aprendido e sai com a conta.

TODAS AS 7 COLUNAS, CÓPIA FIEL DE ARQUITETURA_BANCO_COMPLETA.MD §3, COM EXPLICAÇÃO

Coluna	Tipo	Aceita vazio?	Chave	O que guarda, em linguagem simples
id	uuid	não	PK	Identificador do aprendizado.
user_id	uuid	não	FK → users.id	De quem é essa memória. Aprendizado de um usuário nunca contamina o de outro.
category_name	text	sim	-	Categoria relacionada ao aprendizado, quando faz sentido ('Alimentação').
preference_key	text	não	-	O nome do aprendizado, como uma etiqueta: income_profile, top_categories, payment_style. É por esta chave que o sistema busca.
preference_value	jsonb	não	-	O conteúdo aprendido, em JSON. Formato livre de propósito: cada tipo de aprendizado tem uma estrutura diferente e o banco não precisa mudar para acomodar um novo.
frequency	integer	não	-	Quantas vezes o padrão se repetiu. É o 'peso' da memória: quanto maior, mais confiança o sistema tem naquele palpite.
updated_at	timestampz	não	-	Último reforço desse aprendizado.

Entradas, quem aponta para cá

- users.id → user_id (1:N, CASCADE).

Saídas, para quem esta tabela aponta

- Nenhuma FK de saída.

```
-- Preferências aprendidas de um usuário, das mais fortes para as mais fracas
SELECT preference_key, preference_value, frequency
  FROM financial_memory
 WHERE user_id = $1
 ORDER BY frequency DESC;
```

Exemplo concreto de uso

Na prática, na quinta vez que a pessoa registra 'ifood', a linha com preference_key = 'top_categories' já está com frequency = 5 apontando Alimentação. O parser usa isso como dica e acerta a categoria imediatamente, resposta mais rápida e mais barata.

document_imports

Grupo IA · 10 colunas · Importação de PDFs/extratos pelo painel.

Imagine o protocolo de entrega de documentos de um cartório: cada envelope recebido ganha um número, um status ('em análise', 'concluído') e uma anotação do que saiu dele. A tabela `document_imports` é esse protocolo para os extratos em PDF.

Na prática, ela dá transparência a uma operação que pode demorar e falhar. Ler um PDF de banco envolve extrair texto e pedir à IA para achar os lançamentos, pode dar erro. Em vez de travar a tela, o sistema registra a importação, processa e atualiza o status.

Quem escreve nesta tabela

A rota `POST /imports/pdf` e o `media-processor.ts`, que atualiza o status durante o processamento.

Quem lê esta tabela

A tela de importação do painel (acompanhamento) e a auditoria de quantos lançamentos vieram de arquivo.

Se o registro pai for apagado

CASCADE com `users`.

TODAS AS 10 COLUNAS, CÓPIA FIEL DE ARQUITETURA_BANCO_COMPLETA.MD §3, COM EXPLICAÇÃO

Coluna	Tipo	Aceita vazio?	Chave	O que guarda, em linguagem simples
<code>id</code>	<code>uuid</code>	não	PK	Número de protocolo da importação.
<code>user_id</code>	<code>uuid</code>	não	FK → <code>users.id</code>	Quem enviou o arquivo.
<code>file_name</code>	<code>text</code>	não	-	Nome original do arquivo, para a pessoa reconhecer ('extrato-setembro.pdf').
<code>file_type</code>	<code>text</code>	não	-	Tipo do arquivo (<code>application/pdf</code> , <code>image/jpeg</code>). Define o processador usado.
<code>status</code>	<code>enum import_status</code>	não	-	<code>pending</code> , <code>processing</code> , <code>completed</code> ou <code>failed</code> . É o campo que a tela consulta para mostrar a barrinha de progresso.
<code>extracted_text</code>	<code>text</code>	sim	-	Todo o texto extraído do documento. Fica guardado para reprocessar sem pedir o arquivo de novo.
<code>transactions_created</code>	<code>integer</code>	sim	-	Quantos lançamentos nasceram deste arquivo. É o resultado prático da importação.
<code>metadata</code>	<code>jsonb</code>	sim	-	Detalhes técnicos em JSON: número de páginas, banco identificado, período detectado.
<code>error_message</code>	<code>text</code>	sim	-	Motivo da falha em linguagem clara ('PDF protegido por senha').
<code>created_at</code>	<code>timestampz</code>	não	-	Quando o arquivo foi enviado.

Entradas, quem aponta para cá

- `users.id` → `user_id` (1:N, CASCADE).

Saídas, para quem esta tabela aponta

- Nenhuma FK de saída.

```
-- Importações e o resultado de cada uma
SELECT file_name, status, transactions_created, error_message
FROM document_imports
WHERE user_id = $1
ORDER BY created_at DESC;
```

Exemplo concreto de uso

Na prática, enviar 'extrato-setembro.pdf' cria uma linha com `status = pending`. O sistema extrai o texto, a IA identifica 27 lançamentos, cria as transações e a linha termina com `status = completed` e `transactions_created = 27`. Se o PDF tivesse senha, o status seria `failed` com o motivo escrito.



subscriptions

Grupo Assinatura · 8 colunas · Assinatura Stripe (plano pago).

Imagine o contrato da sua operadora de celular: qual plano, se está em dia e até quando vale o período já pago. A tabela `subscriptions` é a cópia desse contrato dentro do sistema, o original fica no Stripe.

Na prática, ela evita que o Controla.AI dependa da internet do Stripe para responder uma pergunta simples: 'esta pessoa pode usar o recurso pago?'. O webhook do Stripe avisa qualquer mudança e o sistema atualiza esta linha; a consulta do dia a dia é local e instantânea.

Quem escreve nesta tabela

O webhook de billing, ao receber eventos de assinatura criada, renovada, cancelada ou com pagamento recusado.

Quem lê esta tabela

As verificações de plano nas rotas protegidas e a tela administrativa de assinantes.

Se o registro pai for apagado

CASCADE com users. As colunas do Stripe não são FK do banco, são identificadores de um sistema externo.

TODAS AS 8 COLUNAS, CÓPIA FIEL DE ARQUITETURA_BANCO_COMPLETA.MD §3, COM EXPLICAÇÃO

Coluna	Tipo	Aceita vazio?	Chave	O que guarda, em linguagem simples
id	uuid	não	PK	Identificador interno da assinatura.
user_id	uuid	não	FK → users.id	Assinante.
stripe_sub_id	text	sim	-	Código da assinatura no Stripe. Não é chave estrangeira do banco: é o 'número do contrato' no sistema externo.
stripe_price_id	text	sim	-	Código do preço/plano contratado no Stripe.
plan	enum plan	não	-	free, pro ou premium. Repete o plano aqui para consulta rápida, sem precisar ir ao Stripe.
status	enum	não	-	Situação do contrato: ativo, cancelado, inadimplente, em teste.
current_period_end	timestamp tz	sim	-	Até quando o período pago vale. É o que decide se o acesso continua liberado hoje.
created_at	timestamp tz	não	-	Quando a assinatura foi criada.

Entradas, quem aponta para cá

- users.id → user_id (1:N, CASCADE).

Saídas, para quem esta tabela aponta

- Nenhuma FK de saída.

```
-- Assinatura vigente de um usuário
SELECT plan, status, current_period_end
FROM subscriptions
WHERE user_id = $1
AND current_period_end > NOW();
```

Exemplo concreto de uso
Na prática, quando alguém assina o plano pro, o Stripe cobra e dispara o webhook; o backend grava aqui plan = pro, status = active e a data final do período. Se o cartão falhar no mês seguinte, o Stripe avisa e o status muda, o sistema reage sem ninguém mexer no banco à mão.

E

Fluxo 1: “gastei 45 no almoço” pelo WhatsApp

Da frase digitada à linha no banco, com cada tabela tocada

Este é o fluxo mais importante do produto e vale acompanhá-lo com atenção, porque ele atravessa todas as camadas apresentadas até aqui. O usuário escreve cinco palavras no WhatsApp; onze etapas depois, existe uma linha no PostgreSQL e um gráfico atualizado no painel.

- 1

A mensagem chega. O Baileys dispara o evento e o `client.ts` confere o horário: mensagem antiga ou histórico de reconexão é descartado. *Nenhuma tabela tocada.*
- 2

Deduplicação. O identificador da mensagem é comparado com o que já foi processado. *Consulta em `whatsapp_messages.whatsapp_message_id`.*
- 3

Descoberta do telefone. O `jid-resolver.ts` traduz o endereço interno do WhatsApp para o número real. *Nenhuma tabela tocada.*
- 4

Identificação do dono. O `user-resolver.ts` testa variações do número e encontra a pessoa. *SELECT em `users`, sem resultado, o fluxo termina aqui com o link de cadastro.*
- 5

Registro da entrada. A mensagem é gravada com direção inbound. *INSERT em `whatsapp_messages`.*
- 6

Verificações do agente. Não é saudação, não há meta em andamento, não é informação de renda. O caminho segue para o interpretador. *SELECT em `whatsapp_sessions` e `user_settings`.*
- 7

Interpretação pela IA. O parser envia a frase, as categorias do usuário e o histórico recente ao GPT-4o-mini e recebe: intenção transação, tipo despesa, valor 45, categoria Alimentação. *INSERT em `ai_Logs` com `tokens`, `custo` e `tempo`.*
- 8

Resolução da categoria. O `category-resolver.ts` encontra o identificador de Alimentação entre as globais e as personalizadas. *SELECT em `categories`.*
- 9

Gravação do gasto. Nasce a linha com valor 45,00, tipo despesa, origem whatsapp e o texto original preservado. *INSERT em `transactions`.*
- 10

Fechamento do ciclo. O identificador da transação é gravado na mensagem, a chave estrangeira nº 18. *UPDATE em `whatsapp_messages.transaction_id`.*
- 11

Resposta e memória. O usuário recebe “☒ Registrado: R\$ 45,00 em Alimentação” em bolhas; o aprendizado de categoria é reforçado e a resposta fica registrada. *INSERT em `whatsapp_messages` (outbound) e UPDATE em `financial_memory`.*

RESUMO: O QUE ACONTECEU NO BANCO

Tabela	Operação	Para quê
users	SELECT	Descobrir de quem é a mensagem
whatsapp_messages	INSERT ×2 + UPDATE	Registrar entrada, saída e o vínculo com o gasto
whatsapp_sessions · user_settings	SELECT	Verificar contexto e perfil antes de interpretar
ai_logs	INSERT	Auditara chamada de IA com custo e tempo
categories	SELECT	Encontrar a etiqueta correta
transactions	INSERT	Gravar o gasto, o objetivo final
financial_memory	UPDATE	Reforçar o aprendizado de categoria

Na prática

Cinco palavras do usuário mobilizaram sete tabelas, uma chamada de IA auditada e três proteções contra erro. E o resultado é rastreável de ponta a ponta: dá para partir do valor no gráfico, chegar à linha da transação, dela à mensagem original e daí à chamada de IA que a interpretou, com custo e tempo registrados.



Fluxo 2: login no painel e carregamento do dashboard

Do clique em Entrar aos gráficos na tela

O segundo fluxo é o mais usado no dia a dia e atravessa as duas nuvens do projeto. Ele mostra na prática por que a arquitetura foi dividida entre Vercel e Railway.

- 1 **O usuário preenche e envia.** A tela `Login.tsx` chama `POST /api/auth/login` pelo cliente HTTP central.
- 2 **A função da borda atende.** Pela reescrita do `vercel.json`, o pedido vai para `api/auth/login.ts`, executada na Vercel, não na Railway.
- 3 **Conferência da senha.** A função busca o usuário pelo e-mail e usa `bcrypt.compare`. *SELECT em users*. Senha errada devolve 401 sem revelar se o e-mail existe.
- 4 **Emissão do crachá.** Um JWT assinado em HS256, válido por 7 dias, carregando o identificador do usuário. Se o segundo fator estiver ativo, a função pede o código antes de liberar a sessão completa.
- 5 **O navegador guarda.** O contexto de `lib/auth.tsx` armazena o crachá e passa a anexá-lo automaticamente em todo pedido seguinte.
- 6 **O dashboard pede dados.** Ao abrir, a tela dispara várias chamadas em paralelo: resumo do mês, transações recentes, indicadores e metas.
- 7 **O proxy encaminha.** Cada chamada `/api/*` passa por `api/backend-proxy` e chega ao Fastify na Railway.
- 8 **O porteiro confere.** O `authPreHandler` valida a assinatura do crachá e carrega o usuário no contexto do pedido.
- 9 **As consultas rodam.** Cada rota soma o que precisa, **sempre filtrando pelo dono**. *SELECT em transactions, budgets, categories, goals e goal_checkpoints*.
- 10 **A tela monta.** O React recebe os JSONs e desenha cartões, gráfico por categoria, evolução do mês, barras de progresso e a lista de lançamentos.

Por que o login fica na Vercel

Precisa ser rápido e estar disponível mesmo durante uma implantação do servidor principal. Como conversa direto com o banco, não depende do Fastify estar no ar.

Por que o crachá vale 7 dias

Equilíbrio entre conveniência e risco: o usuário não relogar toda hora, mas um crachá roubado deixar de funcionar em uma semana.

Por que várias chamadas em paralelo

Cada pedaço do dashboard carrega independente. Se os indicadores demorarem, os cartões já aparecem, a tela nunca fica totalmente em branco esperando.

Por que filtrar sempre pelo dono

É a garantia estrutural de isolamento. Somada à chave estrangeira no banco, torna o vazamento entre contas um problema de modelagem, não de atenção do programador.

Na prática

O mesmo crachá emitido por uma função na Vercel é validado por um servidor na Railway. Isso só funciona porque a variável `JWT_SECRET` é idêntica nas duas plataformas, é o detalhe de configuração que costuma render a pergunta “e se as duas nuvens discordarem?”. A resposta é: elas compartilham a mesma chave de assinatura.

E Fluxo 3: “quero juntar 5000 em 6 meses”

Conversa de vários turnos até a meta gravada

Este fluxo mostra algo que os anteriores não mostram: uma conversa que **não termina numa mensagem**. Criar meta exige coletar informações que podem vir aos poucos, e por isso o sistema precisa lembrar onde parou, usando a tabela de sessões.

- 1 **O usuário manifesta a intenção.** “Quero criar uma meta” ou direto “quero juntar 5000 em 6 meses”. O `goal-agent.ts` reconhece o pedido antes de qualquer outra etapa do parser financeiro.
- 2 **A separação crítica.** O `goal-parser.ts` extrai **valor** e **prazo** em campos distintos: 5000 vai para o alvo, 6 vai para a duração em meses. Sem essa separação, o sistema criaria uma meta de R\$ 6. Foi um erro real da equipe e a correção foi separar valor e prazo em campos distintos.
- 3 **Cálculo da data limite.** A partir de hoje mais 6 meses nasce o `deadline_at`. É esse campo que fecha a janela de tempo usada para medir o progresso.
- 4 **Se faltar informação, pergunta.** Dito apenas “quero juntar 5000”, o agente pergunta o prazo e guarda o que já sabe. *INSERT ou UPDATE em `whatsapp_sessions` com o estado do fluxo.*
- 5 **A resposta seguinte é entendida em contexto.** Dez minutos depois o usuário responde “6 meses”. O agente lê a sessão, sabe que se trata de prazo de meta e **não** interpreta como gasto. *SELECT em `whatsapp_sessions`.*
- 6 **A meta é gravada.** Nasce a linha com tipo poupança, alvo 5000, duração 6 meses, data limite calculada, alertas de 80% e 100% ativos e situação ativa. *INSERT em `goals`.*
- 7 **A sessão é encerrada.** O estado do fluxo é marcado como concluído, liberando o agente para tratar a próxima mensagem normalmente. *UPDATE em `whatsapp_sessions`.*
- 8 **Progresso e resposta.** O `goals-service.ts` soma as receitas da janela e devolve a porcentagem inicial, com o link do painel. *SELECT em `transactions`.*
- 9 **Acompanhamento mensal.** A cada mês nasce uma foto do progresso, com gasto, limite da época, porcentagem e as marcas de alerta já enviado. *INSERT em `goal_checkpoints`.*
- 10 **Alertas sem repetição.** Ao cruzar 80%, o aviso é enviado e a marca correspondente fica registrada, o usuário não recebe a mesma mensagem cinco vezes. *UPDATE em `goal_checkpoints`.*

Meta de limite versus meta de poupança

Limite usa ciclo mensal, trimestral ou anual e pergunta “quanto já gastei neste ciclo?”. Poupança usa a janela do prazo e pergunta “quanto já juntei do alvo?”. Duas contas diferentes na mesma tabela.

Por que guardar a foto do limite

A coluna `limit_snapshot` registra qual era o limite na época. Se o usuário aumentar o limite depois, o histórico dos meses anteriores continua verdadeiro.

Na prática

A tabela de sessões é o que transforma mensagens isoladas em conversa. Sem ela, cada frase seria interpretada no vácuo e “6 meses” viraria um gasto de R\$ 6. E como o estado fica no banco (não na memória do programa), uma reinicialização do servidor no meio do fluxo não faz o usuário perder o que já informou.

E Fluxo 4: “quanto gastei esse mês?”

A pergunta vira SQL, e o SQL vira resposta em linguagem natural

O último fluxo é o que melhor demonstra a divisão de trabalho entre inteligência artificial e banco de dados. A IA entra duas vezes, para entender a pergunta e para redigir a resposta, mas **o número nunca passa por ela**.

- 1 A pergunta chega.** Pode vir em dezenas de formas: “quanto gastei”, “quanto torrei esse mês”, “tá caro esse mês?”. Todas significam a mesma coisa.
- 2 Triagem rápida.** O `transaction-intent.ts` reconhece por expressão regular que é pergunta, e não registro de gasto, isso evita uma chamada de IA quando o padrão é óbvio.
- 3 Classificação pela IA.** O parser recebe a frase e devolve intenção consulta com `queryType = monthly_spending`. *INSERT em `ai_Logs`*. Note: a IA **não** devolve valor nenhum, só o tipo da pergunta.
- 4 O banco calcula.** O `insights.ts` executa a soma real das despesas do mês corrente, filtrando pelo dono. *SELECT SUM em `transactions`*.
- 5 Contexto agregado.** Para a resposta não ser um número solto, o sistema busca a renda registrada e as categorias que mais consumiram. *SELECT em `budgets` e `categories`*.
- 6 Projeção.** Com os dias restantes do mês e o ritmo de gasto atual, calcula-se uma estimativa de fechamento, informação útil e ainda baseada só em dado real.
- 7 Redação da resposta.** Agora sim a IA volta: ela recebe os **números já calculados** e escreve um texto natural. Ela redige, não calcula. *INSERT em `ai_Logs`*.
- 8 Verificação anti-repetição.** A resposta é comparada com as últimas enviadas; se estiver igual, é reformulada. *SELECT em `whatsapp_messages`*.
- 9 Entrega.** O texto é dividido em bolhas curtas e enviado, com o link do painel para quem quiser ver o gráfico. *INSERT em `whatsapp_messages` (outbound)*.

A CONSULTA QUE REALMENTE PRODUZ O NÚMERO

```
SELECT COALESCE(SUM(t.amount), 0) AS total_gasto,
       COUNT(*)                 AS qtd_lancamentos,
       c.name                    AS categoria
FROM transactions t
LEFT JOIN categories c ON c.id = t.category_id
WHERE t.user_id = $1
      AND t.type = 'expense'
      AND t.occurred_at >= date_trunc('month', NOW())
GROUP BY c.name
ORDER BY total_gasto DESC;
```

Por que a IA não pode calcular

Um modelo de linguagem prevê a próxima palavra; ele não soma. Pedir a conta a ele produziria números plausíveis e errados, o pior tipo de erro num sistema financeiro, porque parece certo.

Por que a IA ainda é essencial

Sem ela, seria preciso prever cada forma de perguntar. Com ela, “quanto torrei” e “gastei muito?” chegam ao mesmo `queryType`, e a resposta sai em tom humano, não em relatório.

Na prática

A regra de ouro do projeto é: **a IA cuida da linguagem, o banco cuida dos números**. Toda resposta financeira tem origem verificável numa consulta SQL, e cada chamada de IA está registrada com custo e tempo. É essa fronteira clara que permite afirmar à banca que nenhum valor exibido ao usuário foi inventado por um modelo.



De qual tabela sai cada número do dashboard

Rastreabilidade completa dos indicadores da tela

Toda informação mostrada no painel tem origem localizável. Esta página faz o caminho inverso do documento: parte do que o usuário vê e aponta a tabela, as colunas e o cálculo que produziram aquele número. É a resposta direta para a pergunta “de onde vem esse valor?”.

Indicador na tela	Tabelas de origem	Colunas usadas	Cálculo
Ganhos do mês	transactions	amount, type, occurred_at, user_id	Soma do valor onde o tipo é receita e a data está no mês corrente
Gastos do mês	transactions	amount, type, occurred_at, user_id	Soma do valor onde o tipo é despesa e a data está no mês corrente
Saldo do mês	transactions + user_settings	amount, type, initial_balance	Ganhos menos gastos, partindo do saldo inicial informado no onboarding
Renda esperada	budgets	total_income_expected, month	Leitura direta da linha do mês corrente
Teto de gastos	budgets	total_expense_limit, month	Leitura direta, comparada com os gastos acumulados
Gráfico por categoria	transactions + categories	amount, category_id, name, color	Agrupar os gastos por categoria e soma cada grupo; a cor vem da categoria
Evolução no mês	transactions	amount, occurred_at	Agrupar por dia e acumula ao longo do período
Maior despesa	transactions + categories	amount, description, occurred_at	Maior valor entre as despesas, com descrição e etiqueta
Comparação com o mês anterior	transactions	amount, type, occurred_at	Dois períodos calculados separadamente e comparados em reais e percentual
Lista de transações	transactions + categories	todas as colunas do lançamento	Listagem filtrada pelo dono, ordenada pela data do fato
Progresso das metas	goals + transactions	limit_amount, target_amount, period_type, goal_type, duration_months, deadline_at	Soma as transações da janela correta e divide pelo alvo ou limite
Histórico das metas	goal_checkpoints	month, spent_amount, limit_snapshot, percentage	Leitura das fotos mensais já calculadas
Contas fixas previstas	recurring_transactions	amount, type, next_due, is_active	Soma o que está ativo e vence no período
Plano da conta	users + subscriptions	plan, status, current_period_end	Plano vigente, validado pela data do período pago

Fonte única de cálculo

Os cartões do topo saem do mesmo cálculo que o assistente usa no WhatsApp. É por isso que os dois canais nunca mostram valores diferentes, não existem duas implementações da mesma soma.

Cores consistentes

A cor de cada categoria vem da coluna `color`, não de uma paleta sorteada na tela. Alimentação tem a mesma cor no gráfico de pizza, na linha do tempo e na lista.

Na prática

Nenhum número do painel é calculado no navegador a partir de dado parcial. Todos vêm de consultas no servidor, filtradas pelo dono e somadas no PostgreSQL. Para a banca, isso permite pegar qualquer indicador da tela e demonstrar, com a consulta em mãos, exatamente de onde ele saiu.

F

De qual tabela sai cada resposta do assistente

As perguntas do usuário e as consultas correspondentes

A página anterior mapeou a tela; esta mapeia a conversa. Para cada pergunta que o assistente sabe responder, aqui estão as tabelas consultadas e o que exatamente é calculado. Junto, ficam as tabelas de apoio que fazem a conversa funcionar sem ser sobre números.

CONSULTAS FINANCEIRAS

Pergunta do usuário	queryType	Tabelas	O que é calculado
“Quanto gastei esse mês?”	monthly_spending	transactions	Soma das despesas do mês, com as principais categorias
“Quais dias gastei mais?”	top_spending_days	transactions	Agrupamento por dia, ordenado do maior para o menor
“Qual foi minha maior despesa?”	biggest_expense	transactions + categories	Maior valor, com descrição, data e etiqueta
“Posso gastar 500?”	can_spend	budgets + transactions + user_settings	Renda menos gastos, mais projeção pelos dias restantes
“Como está minha situação?”	health_check	transactions + budgets + goals	Retrato completo: entradas, saídas, saldo, categorias e metas
“Gastei mais que no mês passado?”	month_comparison	transactions	Dois períodos e a diferença em reais e percentual
“Já cadastrei minha renda?”	income_profile_status	budgets + user_settings	Verifica renda do mês e conclusão do onboarding

TABELAS DE APOIO, O QUE FAZ A CONVERSA FUNCIONAR

Tabela	Papel na conversa	Sem ela, o que aconteceria
whatsapp_messages	Histórico e deduplicação; guarda o vínculo com o gasto gerado	O robô repetiria respostas, processaria mensagens duas vezes e não haveria como provar a origem de um lançamento
whatsapp_sessions	Estado dos fluxos de vários turnos (meta, onboarding, clarificação)	Cada mensagem seria interpretada no vácuo: “6 meses” viraria um gasto de R\$ 6
ai_conversations	Histórico do chat do painel, com contexto de mês	Perguntas de continuidade como “e no mês passado?” não teriam sentido
ai_logs	Auditoria de custo, tempo e status de cada chamada	Seria impossível saber quanto a IA custa nem diagnosticar uma resposta errada
financial_memory	Aprendizado de categorias e perfil de renda	O assistente erraria as mesmas categorias indefinidamente e gastaria mais chamadas de IA
user_settings	Perfil de renda e flags de onboarding	As perguntas de renda se repetiriam para sempre, o pior defeito de experiência que o produto já teve
categories	Lista enviada à IA para classificar	O modelo inventaria nomes de categoria e os relatórios ficariam fragmentados
document_imports	Acompanhamento do processamento de PDFs	O usuário não saberia se a importação funcionou, falhou ou ainda está em andamento

Na prática

As 16 tabelas não estão ali por completude de modelagem: cada uma resolve um problema concreto de produto. Metade sustenta o dinheiro (transações, categorias, orçamento, metas) e a outra metade sustenta a conversa (mensagens, sessões, memória, auditoria). Tirar qualquer uma degrada uma funcionalidade específica e identificável.